

Xebia LMS

Enterprise Software Design Document

System Business Requirements, High-Level & Low-Level Design + 11 Module Specifications

ENTERPRISE-GRADE · HORIZONTALLY SCALABLE · EVENT-DRIVEN

Document	Enterprise Software Design Document (SDD)
System	Xebia LMS (com.xebia.lms)
Contents	System BRD + HLD + LLD, then 11 modules at HLD + LLD
Version	2.0
Date	June 2026
Prepared by	Gs Dwivedi — Solution Architect, Engineering
Tech stack	Next.js 14 · Spring Boot 3.3 / Java 21 · PostgreSQL · Redis · AWS S3
Scale targets	10k+ concurrent users · p95 < 1 s core paths · zero double-booking
Classification	Internal — Confidential

Contents

Part I — System Design

1 Business Requirements (BRD)

2 High-Level Design (HLD) — incl. scaling architecture

3 Low-Level Design (LLD) — cross-cutting patterns

Part II — Module Specifications (HLD + LLD)

- M01 Identity & Access Management and Platform Core
- M02 Workforce & Operations Management (Manager)
- M03 Training Delivery Orchestration (Organiser)
- M04 Course Authoring & Trainer Workspace (Trainer)
- M05 Learning Experience & Engagement (Learner)
- M06 Scheduling & Conflict Resolution Engine
- M07 Assessment & AI Evaluation Engine
- M08 Notification & Integration Services
- M09 Frontend Experience Platform & Design System
- M10 Analytics, Reporting & Insights
- M11 Finance, Billing & Revenue

Module Catalog

The platform is organised into eleven modules, each independently ownable by one squad. Names below are the canonical module names used throughout this document.

ID	Module name	What it does	Team
M01	Identity & Access Management and Platform Core	Authentication (JWT RS256), dynamic MODULE:ACTION RBAC, master data, API gateway and audit — the foundation every module depends on.	Platform
M02	Workforce & Operations Management (Manager)	Onboarding of trainers/orgs/universities, trainer ↔ org assignment, the single approval authority, batch & student enrolment, and oversight.	Control
M03	Training Delivery Orchestration (Organiser)	Conflict-safe training proposals, trainer feedback, test & event creation, batch students, and course → batch curriculum assignment.	Organiser
M04	Course Authoring & Trainer Workspace (Trainer)	Course/module/submodule/content authoring with versioning, assessment authoring, evaluation & AI-score override, and schedule view.	Trainer
M05	Learning Experience & Engagement (Learner)	Course player with resumable progress, timed attempts, attendance, badges/awards/certificates, and learner tickets.	Learner
M06	Scheduling & Conflict Resolution Engine	Trainer slots, Redlock locking, cross-organisation conflict detection, booking lifecycle, and immediate slot release.	Scheduling
M07	Assessment & AI Evaluation Engine	Test/question authoring, timed attempts with auto-lock, AI scoring with human override, and results & performance.	Assessment
M08	Notification & Integration Services	Transactional-outbox email, Google Calendar, reminders and S3 presigned media — reliable asynchronous side-effects.	Notifications
M09	Frontend Experience Platform & Design System	Next.js shell, RBAC-driven dynamic navigation, shared component library and design tokens consumed by every role UI.	Frontend
M10	Analytics, Reporting & Insights	Event-driven aggregation across batches/trainers/learners, scoped dashboards, and scheduled reports/exports.	Data
M11	Finance, Billing & Revenue	Contracts & billing rules, invoicing from confirmed delivery, payments, reconciliation and credit notes.	Finance

System Design

Platform-wide business requirements, architecture and the low-level patterns every module shares — including the scaling model that lets the system serve an enterprise load.

1. Business Requirements (BRD)

1.1 Business Context & Vision

Xebia LMS is an enterprise, role-driven learning platform that manages the full training lifecycle for client organisations and universities — from onboarding trainers and learners, through conflict-free scheduling and delivery, to AI-assisted assessment, engagement tracking, analytics and billing.

The platform replaces fragmented, manually-coordinated training operations with a single system in which permissions are centrally and dynamically controlled, a trainer can never be double-booked across organisations, side-effects (email, calendar, reminders) are delivered reliably, and leadership has real-time insight. It is designed to scale horizontally to an enterprise user base while keeping its core operations fast and correct under contention.

1.2 Goals & Success Metrics

Goal	Metric / target
Single source of truth for identity & permissions	One auth+RBAC service; permission changes effective immediately, no redeploy
Conflict-free scheduling	Zero trainer double-bookings, even under concurrent proposals
Responsive core operations	p95 < 1 s for availability, propose, approve, attempt
Enterprise scale	10,000+ concurrent users; horizontal scale with no architectural change
Reliable side-effects	No lost emails/calendar invites; at-least-once delivery with idempotency
AI-assisted evaluation	Automated first-pass scoring with authoritative human override
Actionable insight	Leadership dashboards refreshed from events, no impact on OLTP
Auditability	Every privileged action recorded immutably

1.3 Scope

In scope

- Identity, authentication and dynamic role-based access control across all roles.
- Workforce operations: onboarding, assignment, approvals, enrolment and oversight.
- Training delivery, course authoring, and the learner experience.
- Conflict-aware scheduling, AI-assisted assessment, and reliable notification/integration.
- Frontend platform & design system, analytics & reporting, and finance & billing.

Out of scope (future)

- Native mobile applications (the platform is responsive web).
- Deep payment-gateway processing (finance records and reconciles; gateway integration is a later phase).
- A public course marketplace / self-service catalogue.

1.4 Stakeholders & Actors

Actor	Description
Super Admin / Admin	Owns roles, modules and permissions; manages users (M01).
Manager	Governs supply and demand; the single approval authority (M02).
Organiser	Runs day-to-day delivery for client organisations (M03).
Trainer	Authors courses and assessments; delivers and evaluates (M04).
Learner	Consumes courses, attempts assessments, tracks progress (M05).
Executive (University / Org head)	Consumes oversight and leadership dashboards (M02, M10).
Finance user	Manages contracts, invoicing and reconciliation (M11).

System workers	Outbox relay, schedulers, ingestion and scoring (cross-cutting).
----------------	--

1.5 Functional Requirement Groups

Detailed, ID-traced functional requirements live in each module specification (Part II). They are grouped here for orientation; each group maps to one module and its FR prefix.

Group	Summary	Module · FR prefix
Identity & Access	Authenticate users; resolve dynamic permissions deny-by-default.	M01 · FR-IAM
Workforce & Operations	Onboard, assign, approve, enrol, oversee.	M02 · FR-MGR
Training Delivery	Propose trainings, feedback, tests, events, curriculum.	M03 · FR-ORG
Course Authoring	Author versioned courses/content; evaluate; override.	M04 · FR-TRN
Learning Experience	Consume courses; attempt; attendance; rewards.	M05 · FR-LRN
Scheduling	Availability, conflict-free booking, lifecycle, release.	M06 · FR-SCH
Assessment	Author/assign/attempt/score with override.	M07 · FR-ASM
Notification	Reliable async email/calendar/reminders/media.	M08 · FR-NTF
Frontend Platform	Dynamic nav, component library, tokens.	M09 · FR-FE
Analytics	Aggregate, dashboard, schedule, export.	M10 · FR-RPT
Finance	Contract, invoice, pay, reconcile.	M11 · FR-FIN

1.6 Key Business Rules (cross-cutting)

#	Rule
BR-G1	Access is denied unless an explicit permission grant exists (deny-by-default).
BR-G2	Only the Manager role may approve a training/event state change.
BR-G3	A trainer is never double-booked across any organisation.
BR-G4	A held or cancelled training releases its slot immediately.
BR-G5	All media is private; access is via short-lived presigned URLs.
BR-G6	Every privileged action is written to an immutable audit log.
BR-G7	All timestamps are stored in UTC; time zones are applied at the edges.
BR-G8	Each module owns its schema; other modules reference by id, never by cross-module foreign key.

1.7 Assumptions & Constraints

- The technology stack (Next.js 14, Spring Boot 3.3 / Java 21, PostgreSQL, Redis, AWS S3) is mandated.
- Deployment is on AWS with multi-AZ availability; containers orchestrated by Docker Swarm / Kubernetes.
- The primary operating region is APMEA; the system is time-zone aware throughout.
- Modules are built by independent squads against published contracts (contracts-first).

2. High-Level Design (HLD)

2.1 Architecture Overview

Xebia LMS is a **layered, event-driven microservices** platform. A Next.js front end is served behind a CDN and load balancer; an API gateway authenticates and routes to stateless application services grouped into a foundation, shared engines, role modules and cross-cutting modules. Services communicate synchronously over REST for queries and asynchronously over a transactional outbox and event bus for side-effects and cross-module propagation. State lives in PostgreSQL (with read replicas), Redis (cache, locks, pub/sub) and S3 (media).

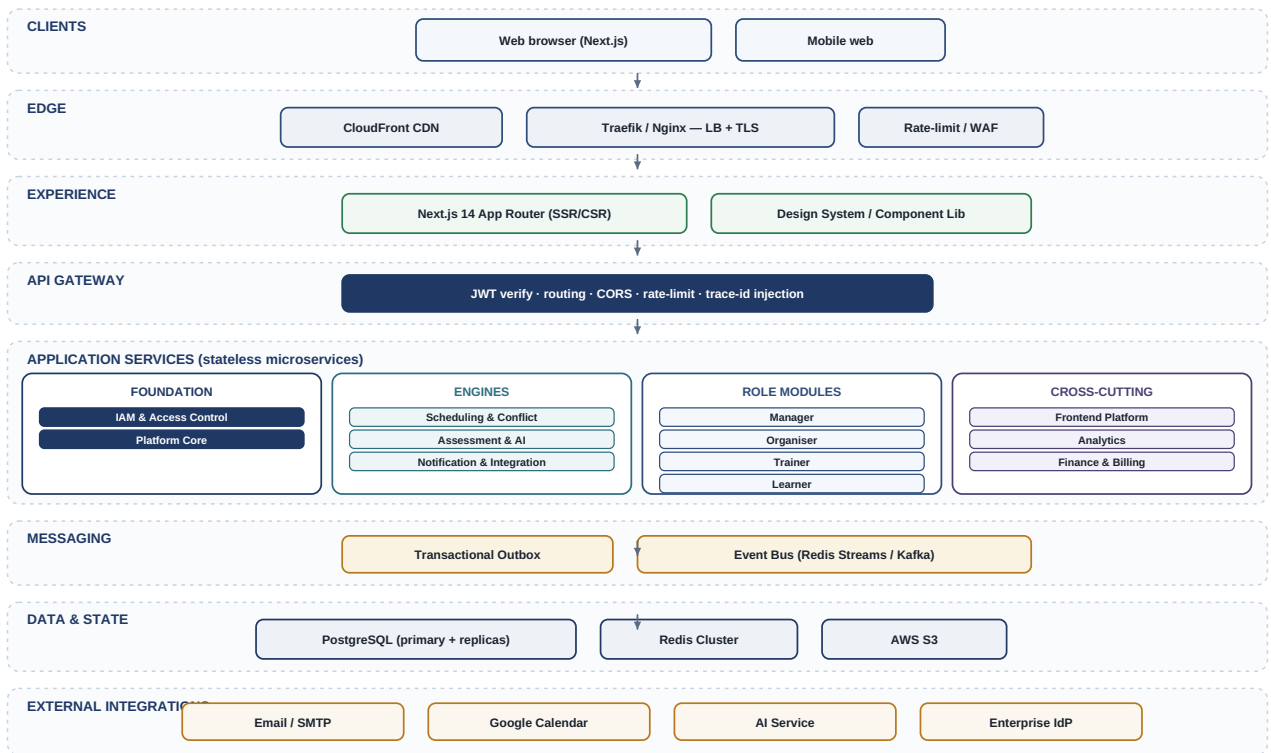


Figure S-1 — Layered system architecture, client edge through data and external integrations.

2.2 Module Landscape

The eleven modules sit in four layers. The foundation and shared engines are platform-wide; role modules are user-facing; cross-cutting modules span the others. Vertical position implies build order: the foundation and engines exist (or are mocked) before role modules go deep.

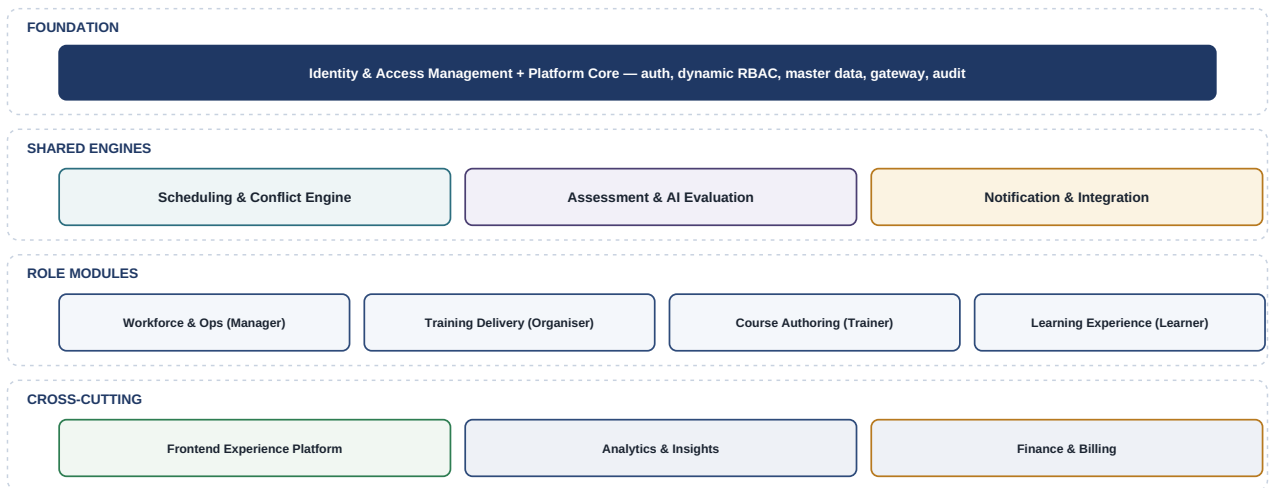


Figure S-2 — Module landscape by layer (canonical names).

2.3 Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router), JavaScript	SSR for SEO/first-paint; one shell, dynamic nav.
Backend	Spring Boot 3.3, Java 21 (virtual threads)	Mature ecosystem; virtual threads for cheap I/O concurrency.
Database	PostgreSQL + Flyway (ddl-auto=none)	ACID, rich indexing; migrations as the schema source of truth.
Cache / locks / pubsub	Redis (cluster)	Sub-ms reads, Redlock, real-time fan-out.
Object storage	AWS S3 + IAM	Durable media; presigned, no public ACL.

Messaging	Transactional outbox + event bus (Redis Streams / Kafka)	Reliable, decoupled async.
Edge	CloudFront CDN + Traefik / Nginx	Static/media caching; TLS, LB, rate-limit.
Delivery	Docker + Swarm / K8s, GitHub Actions	Containerised, autoscaled, CI/CD with contract tests.

2.4 Cross-Cutting Concerns

- **AuthN/AuthZ** — one JWT RS256 + dynamic RBAC service; all modules import the guard (LLD §3.1).
- **Async & consistency** — transactional outbox + event bus; no dual-write (LLD §3.2).
- **Observability** — structured logs, metrics and distributed tracing with a trace id from the gateway.
- **Configuration & flags** — externalised config; feature flags toggle behaviour without redeploy.
- **Error handling** — one error envelope, pagination and idempotency convention (LLD §3.6).

2.5 Scaling & Performance Architecture

The platform is designed to scale **horizontally** with no architectural change. Application services are **stateless and shared-nothing** — any pod serves any request — so capacity is added by adding replicas behind the load balancer, governed by a horizontal pod autoscaler reacting to CPU and request rate. State that must be shared (sessions are not) lives in Redis and PostgreSQL, each scaled on its own axis.

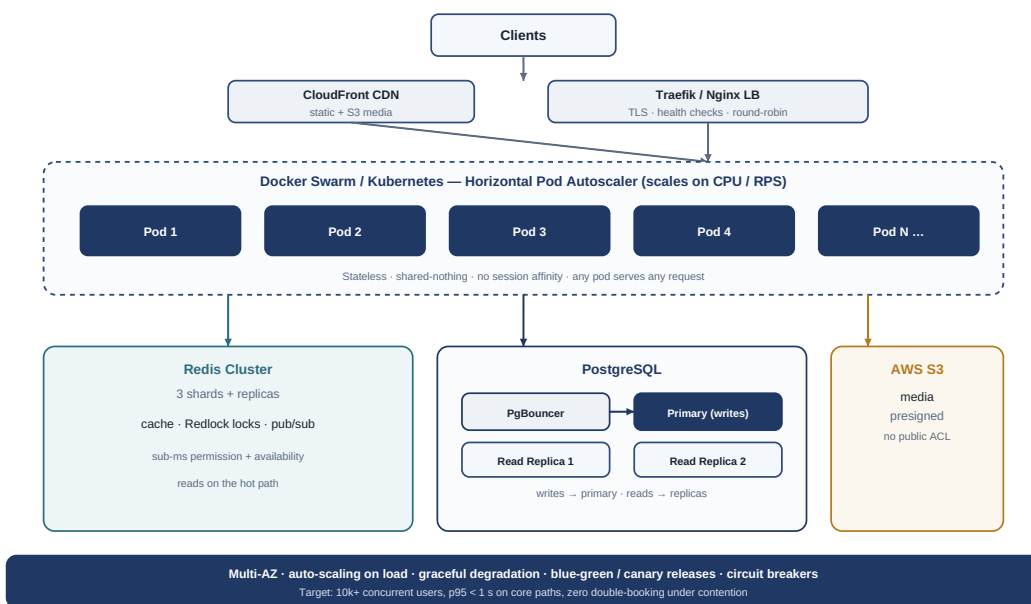


Figure S-3 — Scaling & deployment topology: stateless autoscaled pods over a Redis cluster, PostgreSQL primary/replicas and S3.

Reads dominate the workload, so PostgreSQL runs a **primary for writes and replicas for reads**, fronted by **PgBouncer** connection pooling; analytics reads hit replicas exclusively and never touch the OLTP primary. Redis runs as a **cluster** serving three jobs: caching resolved permissions and availability (sub-millisecond), distributed locks (Redlock) on the scheduling hot path, and pub/sub fan-out for real-time updates. Static assets and media are served via **CloudFront-fronted S3** with short-lived presigned URLs, keeping large payloads off the application tier entirely.

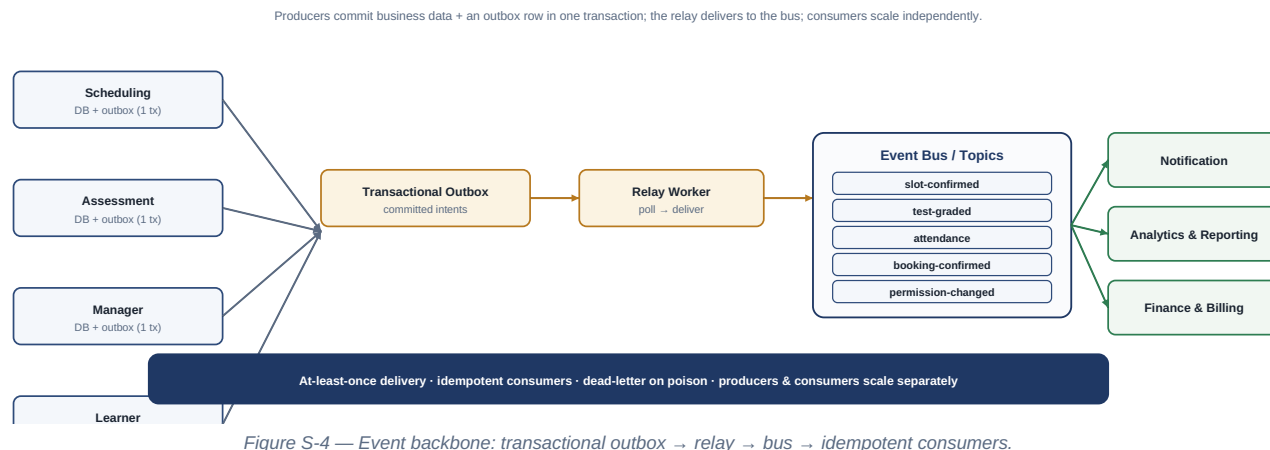
Latency-sensitive correctness — the one hard concurrency requirement — is the scheduling path: a Redis lock serialises concurrent proposals for the same trainer window while a partial unique index in PostgreSQL is the durable backstop, so the system stays both fast and double-booking-free under contention (LLD §3.4). Everything else that can be asynchronous is pushed off the request path through the transactional outbox, so a slow third party never slows a user request.

Concern	Scaling tactic
Application tier	Stateless pods + horizontal autoscaler (CPU/RPS); no session affinity.
Database reads	Read replicas + PgBouncer pooling; read/write split.
Database writes	Single primary; hot tables indexed; partition large/append-only tables (audit, events).
Cache & locks	Redis cluster — permission/availability cache, Redlock, pub/sub.
Media	S3 + CloudFront CDN; presigned URLs keep blobs off the app tier.
Side-effects	Transactional outbox + worker pool scaled on queue depth.
Resilience	Circuit breakers, timeouts, retries with back-off, graceful degradation.
Traffic	Traefik/Nginx LB with health checks; rate-limiting at the gateway.

Capacity target: 10,000+ concurrent users with p95 < 1 s on availability/propose/approve/attempt, sustained while the autoscaler holds CPU below target — validated by load tests in the CI pipeline.

2.6 Event-Driven Architecture

Cross-module communication for state changes is asynchronous and decoupled. A producer commits its business data and an outbox row in a single transaction; a relay delivers committed rows to the event bus; consumers react idempotently and scale independently of producers.



Canonical event catalog

Event	Producer	Consumers	Payload (summary)
slot-confirmed	Scheduling (M06)	Notification, Analytics	bookingId, trainerId, orgId, window
slot-released	Scheduling (M06)	Notification, Analytics	bookingId, reason
booking-confirmed	Scheduling (M06)	Finance, Analytics	bookingId, orgId, batchId, window
test-graded	Assessment (M07)	Learner (rewards), Analytics	attemptId, learnerId, officialScore, domains
attendance-marked	Learner (M05)	Analytics	learnerId, bookingId, status
permission-changed	IAM (M01)	All services, Frontend	userId/roleId, newVersion
feedback-submitted	Organiser (M03)	Notification, Analytics	trainerId, rating, batchId
course-published	Trainer (M04)	Organiser, Analytics	courseId, version

2.7 Security Architecture

- **Transport** — TLS everywhere; the gateway is the only public entry point.
- **Identity** — JWT RS256; the private key never leaves the auth service (KMS-backed).
- **Authorisation** — dynamic MODULE:ACTION RBAC, deny-by-default, enforced server-side on every route.
- **Data isolation** — every query is scoped to the caller's role and organisation.
- **Media** — private S3 with short-lived presigned URLs; no public objects.
- **Auditing** — immutable audit log of privileged actions.
- **Hardening** — OWASP-aligned input validation, rate-limiting, secrets in a vault.

2.8 Data Architecture

Each module owns its schema as a bounded context. Cross-module references are by id (resolved via APIs or events) — there are **no cross-module foreign keys**, which keeps modules independently deployable and the schema independently evolvable. Analytics reads from replicas and derives rebuildable aggregates; it is never a system of record.

Bounded context	Owning module	Primary entities
Identity & RBAC + master data	M01	app_user, role, module, permission, organisation, audit_log
Workforce & enrolment	M02	trainer, trainer_org_link, batch, batch_student, approval_request
Delivery & curriculum	M03	proposal_draft, trainer_feedback, event, course_assignment
Course & content	M04	course, course_module, submodule, content, trainer_evaluation
Learning & engagement	M05	enrolment_progress, attempt_ref, attendance, badge, certificate
Scheduling	M06	training_slot, training_booking, booking_approval, booking_transition
Assessment	M07	test, question, attempt, answer, test_result

Module	Module ID	Database Tables
Notification	M08	outbox, notification_log, calendar_event_link, reminder_job
Frontend config	M09	theme, design_token, nav_item, feature_flag
Analytics (derived)	M10	report_definition, materialised_aggregate, dashboard
Finance	M11	contract, billing_rule, invoice, payment, reconciliation

2.9 Deployment & DevOps

- **Environments** — dev, UAT and production; promotion to main via a release PR with branch protection.
- **CI/CD** — GitHub Actions runs unit + contract tests on every commit and a weekly integration build; blue-green / canary releases.
- **Migrations** — Flyway is the schema source of truth (ddl-auto=none); migrations gate deploys.
- **Containers** — every service is containerised and orchestrated by Swarm / Kubernetes with health checks.

2.10 Availability & Disaster Recovery

- **Multi-AZ** deployment of the app tier, Redis and PostgreSQL.
- **Backups** — automated PostgreSQL backups with point-in-time recovery; S3 versioning.
- **Resilience** — health checks, circuit breakers and graceful degradation (e.g. notifications queue rather than fail the request).
- **Targets** — 99.9% platform availability; the IAM/RBAC critical path targets 99.95%.

3. Low-Level Design (LLD) — Cross-Cutting Patterns

These patterns are implemented once and reused by every module. Per-module entities and methods are in Part II.

3.1 Authentication & Authorisation

Login issues an RS256-signed access JWT (carrying userId, roleId, permissionVersion) and a rotating HttpOnly refresh token. Authorisation resolves the caller's MODULE:ACTION scopes from a version-keyed Redis cache; an admin grant/revoke bumps the version, invalidating the cache instantly.

```
key = "perm:" + userId + ":v" + jwt.permissionVersion
scopes = redis.get(key)
if scopes == null:
    scopes = db.resolveScopes(userId)      # role -> permissions
    redis.set(key, scopes, TTL_15M)
if required not in scopes: -> 403 FORBIDDEN    # deny by default
# grant/revoke: INCR permission_version -> old key abandoned -> instant invalidation
```

3.2 Transactional Outbox

To avoid the dual-write problem (DB commit succeeds, side-effect lost on crash), producers write business data and an outbox row in one transaction; a relay delivers committed rows. The commit is the durable signal.

```
@Transactional
void confirmBooking(id):
    booking.status = CONFIRMED
    save(booking)
    outbox.insert(type=EMAIL, payload=..., idempotencyKey=...) # same tx
# relay (separate): poll committed PENDING rows -> deliver idempotently
# success -> SENT ; transient fail -> back-off retry ; N fails -> dead_letter
```

3.3 Distributed Locking (Redlock)

Concurrent proposals for the same trainer window are serialised by a short-lived Redis lock with a fencing token; only the owner releases it (compare-and-delete via Lua), and the TTL auto-recovers from a crashed holder.

```
token = uuid()
ok = SET ("lock:trainer:"+id+": "+windowBucket) token NX PX 30000
if not ok: -> 423 LOCKED (client retries)
# ... conflict detect + insert booking (+ outbox) ...
if redis.get(key) == token: redis.del(key)      # safe release (Lua CAS)
```

3.4 Concurrency & the Scheduling Hot Path

The lock makes the common path race-free; a database constraint is the durable backstop. The conflict check is an indexed existence

query on the half-open interval, filtered by trainer only (across all organisations).

```
-- backstop: two active bookings on one slot are impossible
CREATE UNIQUE INDEX uniq_active_slot ON training_booking(slot_id)
  WHERE status IN ('PENDING_APPROVAL','CONFIRMED');

-- conflict existence check (overlap), trainer-scoped
SELECT 1 FROM training_booking b JOIN training_slot s ON s.slot_id=b.slot_id
WHERE s.trainer_id=:t AND b.status IN ('PENDING_APPROVAL','CONFIRMED')
  AND s.start_ts < :endTs AND :startTs < s.end_ts LIMIT 1;
```

3.5 Caching Strategy

Tier	Cached	Invalidation
CDN (CloudFront)	Static assets, media	Content-hash / object versioning
App cache (Redis)	Resolved permissions, trainer availability	Version-key bump / short TTL
Query cache (Redis)	Hot read-mostly lookups (master data)	TTL + event-driven eviction

3.6 API Conventions

All APIs are versioned under `/api/v1`, return a uniform error envelope, paginate consistently, and accept an idempotency key on unsafe operations. A trace id is propagated from the gateway.

```
// error envelope
{ "error": { "code": "SLOT_OCCUPIED", "message": "...",
            "traceId": "...", "details": { } } }
// pagination: ?page=0&size=20 -> { items:[...], page, size, total }
// idempotency: header Idempotency-Key: <uuid> on POST/PUT
```

3.7 Bounded Contexts & Referential Strategy

Modules never share tables or cross-module foreign keys. A module resolves another's data by calling its API or by consuming its events and storing the minimum it needs (e.g. Finance stores org/contract references, not the org schema). This keeps each module independently deployable and scalable, and is what makes the contracts-first parallel build possible.

PART II

Module Specifications

Each of the eleven modules at HLD + LLD depth — architecture, components, interfaces and data flow, then entities & fields, service methods, APIs, algorithms and state.

M01 — Identity & Access Management and Platform Core

This module is the platform foundation. It authenticates every user, resolves the dynamic, admin-controlled permissions that decide what each role can see and do, and owns the shared master data (organisations, universities, branches, domains), the API gateway, and the audit log. Every other module depends on it, so it ships first and publishes its contracts before anyone else builds.

Owning team: Platform.

Scope

- **Authentication** — login, token issue/refresh, optional enterprise SSO exchange.
- **Dynamic RBAC** — admin-defined modules, submodules and MODULE:ACTION permissions mapped to roles, resolved at runtime with deny-by-default.
- **Admin console (RBAC administration)** — register modules/submodules, map them to roles with CRUD scopes, manage users, grant/revoke reversibly.
- **Master data & shared domain** — organisation, university, branch and domain taxonomy with read APIs others consume.
- **Platform cross-cutting** — API gateway/routing, common error envelope, pagination, tracing, and the immutable audit log.

HLD — Architecture & Context

The module is a set of Platform-owned services behind the API gateway: an Auth service (tokens, sessions), an RBAC service (modules, permissions, resolution + cache), a Master-data service (organisation/university/branch/domain), and shared libraries (guard middleware, error envelope, tracing). State lives in PostgreSQL with a Redis cache for resolved permissions; the gateway fronts everything and verifies tokens before routing.

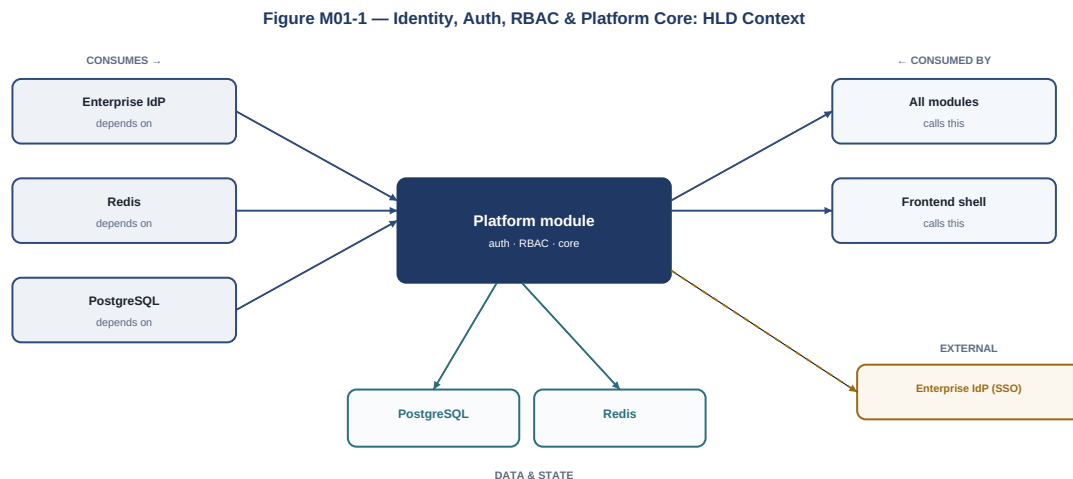


Figure M01-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Auth Service	Login, JWT issue/verify, refresh rotation, SSO exchange, logout/revocation.
RBAC Service	Module/permission registry, role mapping, runtime permission resolution, version-keyed Redis cache, grant/revoke.
Master-data Service	CRUD + read APIs for organisation, university, branch, domain taxonomy.
API Gateway	TLS, routing, JWT verification, CORS, rate-limit, trace-id injection.
Audit Service	Append-only audit log of privileged actions; query API for compliance.
Shared libraries	RBAC guard middleware, error envelope, pagination, tracing — imported by all modules.

HLD — Interfaces

Provides

- POST /auth/login, POST /auth/refresh, POST /auth/logout — token lifecycle.
- GET /me/permissions — resolved scopes for dynamic UI navigation.
- RBAC guard library/middleware (import) and JWKS endpoint for token verification.

- Master-data read APIs: GET /orgs, /universities, /branches, /domains.
- Audit-log write API/lib and query API.
- Event: permission-changed (so caches and UIs refresh).

Consumes

- Enterprise IdP (OIDC/SAML) — optional, for SSO.
- Redis — permission cache and token/session state.
- PostgreSQL — system of record for identity, RBAC and master data.

HLD — Request & Data Flow

- Client → **Gateway**: verify JWT, inject trace id, route.
- Login → **Auth Service**: validate credentials → issue access JWT + HttpOnly refresh cookie.
- Any request → **RBAC guard**: read perm:{userId}:v{ver} from Redis; on miss, resolve from PostgreSQL and cache (TTL).
- Admin grant/revoke → **RBAC Service**: persist mapping → INCR permission version → emit permission-changed → next request re-resolves.
- Other modules → **Master-data read APIs** for org/university/branch/domain by id.

Scaling & performance. Performance: RBAC check < 100 ms (cached); token issue < 150 ms. **Scales by:** Stateless services; Redis-cached resolution scales with pod count.

HLD — Technology

Layer	Technology / choice
Auth	Spring Security, JWT RS256, BCrypt
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

app_user

A platform user of any role.

Field	Type	Notes
user_id	uuid	PK
email	varchar(255)	UNIQUE, login id
password_hash	varchar(100)	BCrypt; null if SSO-only
display_name	varchar(120)	
role_id	uuid	FK → role
org_id	uuid	FK → organisation, nullable
timezone	varchar(64)	IANA tz, e.g. Asia/Kolkata
status	varchar(16)	ACTIVE SUSPENDED INVITED
sso_subject	varchar(255)	external IdP subject, nullable
permission_version	int	bumped on grant/revoke
created_at	timestampz	
last_login_at	timestampz	nullable

role

A named role (Admin, Manager, Organiser, Trainer, Learner, ...).

Field	Type	Notes
-------	------	-------

role_id	uuid	PK
name	varchar(64)	UNIQUE
description	varchar(255)	
is_system	boolean	system roles cannot be deleted
created_at	timestampz	

module

A feature area registered by the Admin.

Field	Type	Notes
module_id	uuid	PK
key	varchar(64)	UNIQUE machine key, e.g. SCHEDULING
title	varchar(120)	
icon	varchar(64)	
route	varchar(160)	
sort_order	int	
is_active	boolean	

submodule

A capability within a module.

Field	Type	Notes
submodule_id	uuid	PK
module_id	uuid	FK → module
key	varchar(64)	
title	varchar(120)	
icon	varchar(64)	

permission

An atomic MODULE:ACTION permission.

Field	Type	Notes
permission_id	uuid	PK
module_id	uuid	FK → module
action	varchar(32)	VIEW CREATE UPDATE DELETE DECIDE
code	varchar(96)	UNIQUE, e.g. SCHEDULING:CREATE

role_permission

Maps a permission to a role (the grant).

Field	Type	Notes
id	uuid	PK
role_id	uuid	FK → role, IDX
permission_id	uuid	FK → permission
granted_by	uuid	FK → app_user
granted_at	timestampz	
revoked_at	timestampz	nullable

refresh_token

Rotating refresh-token record.

Field	Type	Notes
token_id	uuid	PK
user_id	uuid	FK → app_user, IDX
token_hash	varchar(128)	
expires_at	timestampz	

rotated_from	uuid	nullable
revoked	boolean	

organisation

Client organisation or institution (master data).

Field	Type	Notes
org_id	uuid	PK
name	varchar(160)	
type	varchar(24)	UNIVERSITY CORPORATE
branch	varchar(120)	nullable
status	varchar(16)	
created_at	timestampz	

audit_log

Append-only record of privileged actions.

Field	Type	Notes
audit_id	bigserial	PK
actor_id	uuid	FK → app_user, IDX
action	varchar(96)	
target_type	varchar(48)	
target_id	varchar(64)	
reason	varchar(255)	nullable
metadata	jsonb	
created_at	timestampz	IDX

LLD — Service Methods

AuthService

```
TokenPair login(String email, String password)
```

Validates credentials, issues an access JWT and a rotating refresh token; records last_login_at.

```
TokenPair refresh(String refreshToken)
```

Validates and rotates the refresh token, issues a new access token; revokes the old refresh token.

```
void logout(UserId userId, String refreshToken)
```

Revokes the presented refresh token (and optionally all sessions).

```
Jwt issueAccessToken(AppUser user)
```

Builds an RS256-signed JWT carrying userId, roleId and permissionVersion.

RbacService

```
Set<String> resolvePermissions(UserId userId)
```

Resolves effective MODULE:ACTION codes for the user's role; caches under perm:{userId}:v{ver}.

```
boolean hasPermission(UserId userId, String code)
```

Deny-by-default check used by the guard on every route.

```
void grant(RoleId role, PermissionId perm, UserId by)
```

Adds a role → permission mapping, writes audit, bumps affected users' permission version.

```
void revoke(RoleId role, PermissionId perm, UserId by)
```

Soft-revokes the mapping, writes audit, bumps version so the change is instant.

```
Module registerModule(ModuleSpec spec)
```

Registers a module/submodule definition for dynamic navigation.

```
void bumpPermissionVersion(RoleId role)
```

Increments permission_version for all users in the role, invalidating their cache.

MasterDataService

```
List<Organisation> listOrganisations(Filter f)
```

Paginated read of organisations for other modules.

```
Domain getDomain(DomainId id)
```

Returns a domain taxonomy node by id.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/auth/login	Authenticate, issue tokens.	public
POST /api/v1/auth/refresh	Rotate tokens.	public (cookie)
POST /api/v1/auth/logout	Revoke session.	authenticated
GET /api/v1/me/permissions	Resolved scopes for the UI.	authenticated
POST /api/v1/admin/modules	Register a module/submodule.	ADM:RBAC:MANAGE
POST /api/v1/admin/roles/{id}/permissions	Grant a permission to a role.	ADM:RBAC:MANAGE
DELETE /api/v1/admin/roles/{id}/permissions/{permId}	Revoke a permission.	ADM:RBAC:MANAGE
GET /api/v1/orgs	List organisations (master data).	authenticated
GET /api/v1/audit	Query the audit log.	ADM:AUDIT:VIEW

LLD — Key Logic & Algorithms

Runtime permission resolution with versioned cache

Every request resolves the caller's scopes in sub-millisecond time without hitting the database on the hot path.

```
key    = "perm:" + userId + ":v" + user.permissionVersion
scopes = redis.get(key)
if scopes == null:
    scopes = db.resolveScopes(userId)      # role -> permissions
    redis.set(key, scopes, TTL_15M)
if required not in scopes: -> 403 FORBIDDEN # deny by default
# Admin grant/revoke: INCR permission_version -> old key abandoned -> instant invalidation
```

LLD — State / Lifecycle

From → To	Trigger	Effect
INVITED → ACTIVE	User accepts invite / first login	Credentials/SSO linked; permissions resolved.
ACTIVE → SUSPENDED	Admin suspends	Tokens revoked; access denied at gateway.
SUSPENDED → ACTIVE	Admin reinstates	Permissions re-resolved on next login.

LLD — Non-Functional Targets

Attribute	Target / approach
Security	RS256 JWT, BCrypt, deny-by-default, full audit; OWASP-aligned input validation.
Performance	RBAC check < 100 ms (cached); token issue < 150 ms.
Availability	99.95% — this is on the critical path for every request.

Scalability	Stateless services; Redis-cached resolution scales with pod count.
-------------	--

M02 — Workforce & Operations Management (Manager)

The Manager module governs supply and demand across the platform. It onboards trainers, organisations and universities, assigns trainers to organisations and domains, approves or rejects scheduling and event requests (the Manager is the single approver in the platform), manages batch and student enrolment, and gives leadership oversight of delivery. It is the control plane that the Organiser and Trainer modules route decisions through.

Owning team: Control.

Scope

- **Onboarding** of trainers, organisations and universities with their profiles and domains.
- **Assignment** of trainers to organisations/domains, creating the trainer ↔ org links scheduling relies on.
- **Approvals** — decide on training proposals, holds, cancellations and events (approve / reject / hold).
- **Enrolment** — create batches and add students batch-wise with branch filtering.
- **Oversight** — dashboards across trainers, batches, performance and pending decisions.

HLD — Architecture & Context

The Manager module is a Control-squad service plus its Next.js UI. It orchestrates other services rather than owning heavy logic: onboarding/assignment write profile and link records, approvals call the Scheduling and Assessment APIs, enrolment manages batches/students, and oversight reads from replicas and analytics. It is a Phase-2 module that builds against the Scheduling and Assessment mocks.

Figure M02-1 — Manager — Workforce & Operations: HLD Context

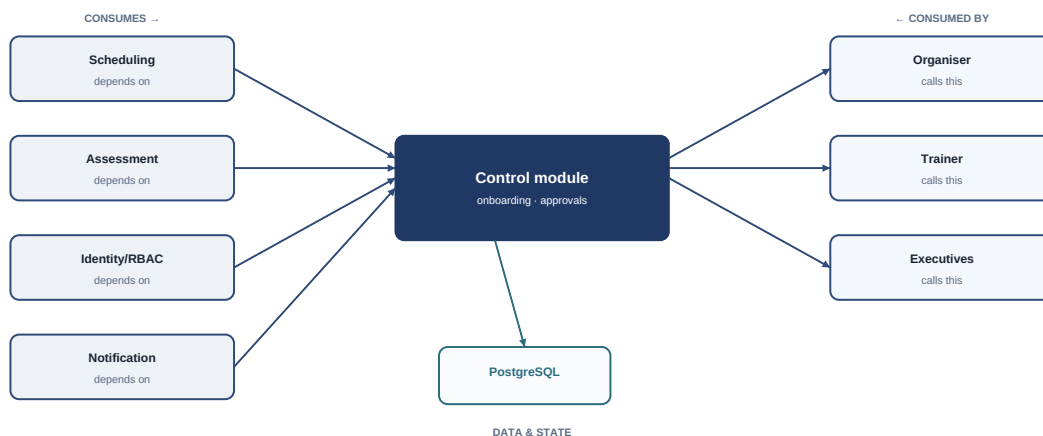


Figure M02-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Onboarding Service	Create/maintain trainer, organisation and university profiles and domains.
Assignment Service	Trainer ↔ organisation/domain links; reassignment and deactivation.
Approval Service	Pending-decision queue; approve/reject/hold orchestration into Scheduling and events.
Enrolment Service	Batch creation and batch-wise student management with branch filter.
Oversight Service	Aggregated read views for managers/executives.

HLD — Interfaces

Provides

- Onboarding & assignment APIs (trainer/org/university, links).
- Approval-queue and decision APIs.
- Enrolment APIs (batch, add students) — consumed by the Organiser.
- Event: trainer-assigned, batch-created.

Consumes

- Scheduling — approve/hold/cancel and read pending proposals.
- Assessment — context for event/test decisions.
- Identity/RBAC — auth, scopes, master data.
- Notification — decision emails (via outbox).

HLD — Request & Data Flow

- Onboard trainer/org → **Onboarding Service** writes profile → emits trainer-assigned where relevant.
- Proposal awaiting decision → appears in **Approval Service** queue (pulled from Scheduling).
- Manager approves → **Approval Service** calls Scheduling approve() → booking CONFIRMED → outbox notifications.
- Create batch + add students → **Enrolment Service** with branch filter → batch ready for Organiser.
- Oversight view → **Oversight Service** reads replicas + analytics events.

Scaling & performance. **Performance:** Approval decision round-trip < 1 s including engine call. **Scales by:** Stateless; queue reads from replicas.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

trainer

An onboarded trainer (may serve many orgs).

Field	Type	Notes
trainer_id	uuid	PK
user_id	uuid	FK → app_user
name	varchar(120)	
type	varchar(24)	INTERNAL EXTERNAL
timezone	varchar(64)	
status	varchar(16)	ACTIVE INACTIVE
bio	text	nullable
created_at	timestampz	

trainer_org_link

Associates a trainer with an organisation and domain (FR-MGR-003).

Field	Type	Notes
id	uuid	PK
trainer_id	uuid	FK, IDX
org_id	uuid	FK
domain_id	uuid	FK
assigned_by	uuid	FK → app_user
assigned_at	timestampz	
active	boolean	UNIQUE(trainer_id,org_id,domain_id) where active

university

An onboarded university (institution).

Field	Type	Notes
university_id	uuid	PK
name	varchar(160)	
branch	varchar(120)	
status	varchar(16)	
created_at	timestamptz	

batch

A cohort within an organisation/university.

Field	Type	Notes
batch_id	uuid	PK
org_id	uuid	FK, nullable
university_id	uuid	FK, nullable
domain_id	uuid	FK
name	varchar(120)	
branch	varchar(120)	
status	varchar(16)	PLANNED RUNNING CLOSED
created_by	uuid	FK → app_user
created_at	timestamptz	

batch_student

A learner enrolled in a batch (branch-scoped).

Field	Type	Notes
id	uuid	PK
batch_id	uuid	FK, IDX
learner_id	uuid	FK → app_user
branch	varchar(120)	
enrolled_by	uuid	FK
enrolled_at	timestamptz	
status	varchar(16)	ENROLLED DROPPED · UNIQUE(batch_id,learner_id)

approval_request

A pending decision routed to the Manager.

Field	Type	Notes
approval_id	uuid	PK
type	varchar(24)	TRAINING HOLD CANCEL EVENT
ref_id	uuid	booking/event id
submitted_by	uuid	FK → app_user
status	varchar(16)	PENDING APPROVED REJECTED HELD
decided_by	uuid	FK, nullable
reason	varchar(255)	nullable
created_at	timestamptz	IDX
decided_at	timestamptz	nullable

assignment

A unit of work assigned to a trainer (task/training pointer).

Field	Type	Notes
assignment_id	uuid	PK
trainer_id	uuid	FK
org_id	uuid	FK

kind	varchar(24)	TRAINING TASK
ref_id	uuid	
created_at	timestamp	

LLD — Service Methods

OnboardingService

```
Trainer onboardTrainer(TrainerForm f)
```

Creates the trainer profile and linked app_user; sets domains and time zone.

```
Organisation onboardOrganisation(OrgForm f)
```

Creates an organisation/university with branches.

```
void deactivateTrainer(TrainerId id, String reason)
```

Marks a trainer inactive and deactivates their assignments.

AssignmentService

```
TrainerOrgLink assign(TrainerId t, OrgId o, DomainId d, UserId by)
```

Creates an active trainer ↔ org/domain link (idempotent on the unique key).

```
void unassign(LinkId id, UserId by)
```

Deactivates a link; future scheduling for that org is blocked.

ApprovalService

```
List<ApprovalRequest> queue(ManagerId m, Filter f)
```

Returns pending decisions with full context for the manager.

```
void approve(ApprovalId id, ManagerId m, String reason)
```

Approves; for TRAINING calls Scheduling.approve(); records decision + audit.

```
void reject(ApprovalId id, ManagerId m, String reason)
```

Rejects; releases any reserved slot via Scheduling; notifies submitter.

```
void hold(ApprovalId id, ManagerId m, String reason)
```

Places on hold; Scheduling releases the slot immediately.

EnrolmentService

```
Batch createBatch(BatchForm f)
```

Creates a batch within an org/university and domain.

```
List<BatchStudent> addStudents(BatchId b, List<LearnerId> ids, String branch)
```

Adds students filtered to the selected branch; dedupes on (batch, learner).

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/trainers	Onboard a trainer.	MGR:ONBOARD:MANAGE
POST /api/v1/orgs	Onboard an organisation/university.	MGR:ONBOARD:MANAGE
POST /api/v1/trainers/{id}/assignments	Assign trainer to org/domain.	MGR:ASSIGN:MANAGE
GET /api/v1/approvals	Pending approval queue.	MGR:APPROVAL:DECIDE

POST /api/v1/approvals/{id}/approve	Approve a request.	MGR: APPROVAL: DECIDE
POST /api/v1/approvals/{id}/reject	Reject a request.	MGR: APPROVAL: DECIDE
POST /api/v1/approvals/{id}/hold	Hold a request.	MGR: APPROVAL: DECIDE
POST /api/v1/batches	Create a batch.	MGR: ENROL: MANAGE
POST /api/v1/batches/{id}/students	Add students (branch-filtered).	MGR: ENROL: MANAGE

LLD — Key Logic & Algorithms

Approval orchestration

The Manager decision is a thin orchestration over the owning engine; the engine enforces side-effects.

- Pull the pending request and its full context (proposal, trainer, batch).
- On approve: call Scheduling.approve(bookingId) → booking CONFIRMED; record decision + reason; audit.
- On reject/hold: call Scheduling.hold/cancel → slot released immediately; notify submitter via outbox.
- Update the approval_request status and decided_at; emit a decision event for analytics.

LLD — State / Lifecycle

From → To	Trigger	Effect
PENDING → APPROVED	Manager approves	Scheduling confirms booking; notifications sent.
PENDING → REJECTED	Manager rejects	Reserved slot released; submitter notified.
PENDING → HELD	Manager holds	Slot released immediately; can be resubmitted.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Approval decision round-trip < 1 s including engine call.
Auditability	Every decision stored with actor, reason, timestamp.
Scalability	Stateless; queue reads from replicas.
Usability	Single approval queue with full context to minimise decision time.

M03 — Training Delivery Orchestration (Organiser)

The Organiser module is the operational owner of training delivery for a client organisation. It proposes trainings into trainers' free time (conflict-safe via the Scheduling engine), submits trainer feedback, creates tests and reads results (via the Assessment engine), manages students batch-wise, and publishes events. A full standalone BRD/HLD/LLD exists for this module; this spec is the team handoff summary with its owned data and methods.

Owning team: Organiser.

Scope

- Dashboard of running courses, assigned trainers and batches scoped to the Organiser's organisations.
- Availability-aware training proposals (mode + venue) submitted for Manager approval.
- Trainer feedback routed to the trainer and the Manager.
- Test creation/assignment and results viewing (via Assessment).
- Batch-wise student addition with branch filter (via Enrolment) and event publishing.

HLD — Architecture & Context

The Organiser module is a Next.js experience plus a thin orchestration service. It calls Scheduling for availability/propose/hold/cancel, Assessment for tests/results, Enrolment for students, and Notification for feedback/event mails. It owns only feedback and event data; everything else is referenced by id. It starts in Phase 1 against the engine mocks and wires the real engines in Phase 2.

Figure M03-1 — Organiser — Training Delivery: HLD Context

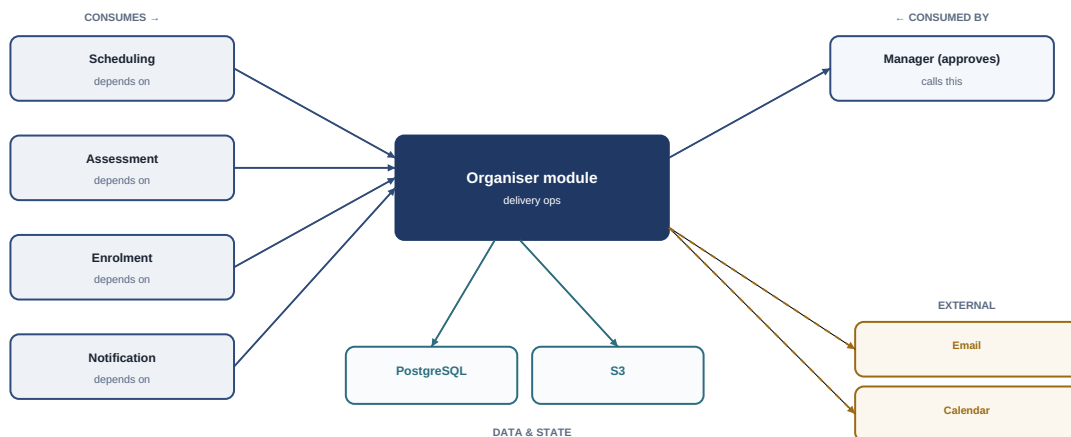


Figure M03-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Organiser BFF	Dashboard composition and proposal/feedback/event orchestration.
Feedback Service	Capture trainer feedback and route to trainer + Manager.
Event Service	Create events with media carousel; submit for approval; publish on approval.
Proposal Orchestrator	Drive the availability → propose → submit flow via Scheduling.

HLD — Interfaces

Provides

- Organiser dashboards and proposal/feedback/event UIs.
- Event: feedback-submitted, event-published.

Consumes

- Scheduling — availability, propose, hold, cancel.
- Assessment — create/assign tests, read results.
- Manager/Enrolment — add students to batches.
- Notification — feedback/event emails + S3 media.

HLD — Request & Data Flow

- Open proposal wizard → call Scheduling availability → pick free slot, set mode/venue.
- Submit → Scheduling.propose() (PENDING) → Manager queue; on approve → confirmations.
- Submit feedback → Feedback Service persists → Notification routes to trainer + Manager.
- Create event → Event Service stores media on S3 → submit for approval → publish on approval with notifications.
- Create test → Assessment.createTest()/assign(); read results via Assessment.
- Assign curriculum → **CurriculumService** maps a published course (version) to a batch → learners see it as an assigned course (Learner reads course_assignment).

Scaling & performance. Performance: Wizard availability < 400 ms; proposal submit < 1 s. **Scales by:** Stateless; reads scoped to organisation.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

proposal_draft

A training proposal being built before submission to Scheduling.

Field	Type	Notes
draft_id	uuid	PK
organiser_id	uuid	FK → app_user
trainer_id	uuid	FK
org_id	uuid	FK
batch_id	uuid	FK
domain_id	uuid	FK
start_ts	timestampz	UTC
end_ts	timestampz	UTC
mode	varchar(12)	ONLINE ONSITE
venue	jsonb	{room,floor,location}
status	varchar(16)	DRAFT SUBMITTED

trainer_feedback

Feedback the Organiser gives on a trainer (FR-ORG-002).

Field	Type	Notes
feedback_id	uuid	PK
trainer_id	uuid	FK, IDX
organiser_id	uuid	FK
batch_id	uuid	FK, nullable
rating	int	1–5
notes	text	
routed_to_manager	boolean	always true
created_at	timestampz	

event

An event/hackathon proposed by the Organiser (FR-ORG-011).

Field	Type	Notes
event_id	uuid	PK
organiser_id	uuid	FK
org_id	uuid	FK
title	varchar(160)	
description	text	
starts_at	timestampz	
ends_at	timestampz	
status	varchar(16)	DRAFT PENDING APPROVED REJECTED PUBLISHED
created_at	timestampz	

event_banner

A carousel item (banner/image/text) for an event (FR-ORG-013).

Field	Type	Notes
banner_id	uuid	PK
event_id	uuid	FK, IDX
kind	varchar(12)	IMAGE TEXT
s3_key	varchar(255)	nullable for TEXT
caption	varchar(255)	
sort_order	int	

event_attachment

A file attached to event notifications.

Field	Type	Notes
attachment_id	uuid	PK
event_id	uuid	FK
s3_key	varchar(255)	
filename	varchar(160)	
size_bytes	bigint	

student_add_request

A batch-wise add-students action (branch-filtered).

Field	Type	Notes
request_id	uuid	PK
batch_id	uuid	FK
branch	varchar(120)	
learner_ids	jsonb	array
requested_by	uuid	FK
created_at	timestampz	

course_assignment

Maps a published course (version) to a batch — the curriculum a cohort follows. Closes the course → batch seam: the Trainer authors/publishes (M04), the Organiser assigns here, the Learner reads it as an assigned course (M05).

Field	Type	Notes
assignment_id	uuid	PK
batch_id	uuid	FK, IDX
course_id	uuid	FK → course (Trainer); UNIQUE(batch_id,course_id)
course_version	int	pinned version at assignment time
organiser_id	uuid	FK → app_user

starts_on	date	
status	varchar(16)	ASSIGNED ACTIVE COMPLETED
assigned_at	timestampz	

LLD — Service Methods

OrganiserService

```
AvailabilityView availability(TrainerId t, DateRange r)
```

Calls Scheduling to fetch the trainer's free slots for the wizard.

```
ProposalResult submitProposal(ProposalDraft d)
```

Validates and calls Scheduling.propose(); returns PENDING booking or 409 with suggestions.

```
void requestHold(BookingId id, String reason)
```

Routes a hold request to the Manager via Scheduling.

```
void requestCancel(BookingId id, String reason)
```

Routes a cancel request to the Manager via Scheduling.

```
DashboardView dashboard(OrganiserId o, Filter f)
```

Composes running courses, trainers, batches and performance, scoped to the org.

FeedbackService

```
TrainerFeedback submit(FeedbackForm f)
```

Persists feedback and enqueues delivery to the trainer (email+in-app) and the Manager.

EventService

```
Event create(EventForm f)
```

Creates an event with banner carousel and attachments on S3; status DRAFT.

```
void submitForApproval(EventId id)
```

Sends the event to the Manager queue (PENDING).

```
void publish(EventId id)
```

On approval, publishes to related trainers/learners and sends notifications.

TestFacade

```
Test createTest(TestForm f)
```

Delegates to Assessment.createTest() and assigns to a batch.

```
ResultsView results(TestId id, Filter f)
```

Reads results/performance from Assessment per learner or batch.

CurriculumService

```
CourseAssignment assignCourse(BatchId b, CourseId c, int version, OrganiserId o)
```

Assigns a published course version to a batch as its curriculum; learners in the batch then see it as an assigned course (Learner module reads this).

```
void activate(AssignmentId id)
```

Activates the assigned curriculum for the batch (status ACTIVE).

```
void completeForBatch(BatchId b, CourseId c)
```

Marks the batch's course curriculum complete.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
GET /api/v1/org/availability	Trainer free slots for the wizard.	ORG:TRAINING:CREATE
POST /api/v1/org/proposals	Submit a training proposal.	ORG:TRAINING:CREATE
POST /api/v1/org/bookings/{id}/hold	Request a hold.	ORG:TRAINING:CREATE
POST /api/v1/org/bookings/{id}/cancel	Request a cancel.	ORG:TRAINING:CREATE
POST /api/v1/org/feedback	Submit trainer feedback.	ORG:FEEDBACK:CREATE
POST /api/v1/org/tests	Create/assign a test.	ORG:TEST:CREATE
GET /api/v1/org/tests/{id}/results	Read results/performance.	ORG:TEST:CREATE
POST /api/v1/org/events	Create an event.	ORG:EVENT:CREATE
POST /api/v1/org/batches/{id}/students	Add students (branch filter).	ORG:STUDENT:MANAGE

LLD — Key Logic & Algorithms

Conflict-safe proposal (delegated)

The Organiser never computes conflicts; it orchestrates the Scheduling engine.

- Fetch availability from Scheduling for the trainer and range.
- User picks a free slot and sets mode/venue; submit the draft.
- Scheduling.propose() acquires a lock, runs conflict detection and creates a PENDING booking, or returns 409 + free suggestions.
- On 409, present the suggestions and let the user retry; on success, show 'awaiting Manager approval'.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Wizard availability < 400 ms; proposal submit < 1 s.
Consistency	No double-booking (guaranteed by Scheduling).
Usability	Clear busy/free messaging; carousel event composer.
Scalability	Stateless; reads scoped to organisation.

M04 — Course Authoring & Trainer Workspace (Trainer)

The Trainer module lets trainers author course content, create assessments (via the Assessment engine), manage the batches assigned to them, evaluate learner submissions and give feedback, and see their schedule with reminders. It owns the course/content domain and consumes the Scheduling and Assessment engines.

Owning team: Trainer.

Scope

- Course authoring — modules, submodules and content (text/code/image/PDF/video).
- Assessment authoring via the engine; review/override of AI scores.
- Assigned-batch views and learner rosters.
- Tasks and tickets raised to/by the trainer.
- Evaluation and learner feedback; schedule view with reminders.

HLD — Architecture & Context

The Trainer module is a Next.js authoring experience plus a Content/Authoring service. It owns course/module/submodule/content and evaluation data; it calls Assessment for test authoring and override, Scheduling for the read-only schedule, and Notification for media presign and reminders. Phase-1 build against the Assessment and Scheduling mocks.

Figure M04-1 — Trainer — Authoring & Delivery: HLD Context

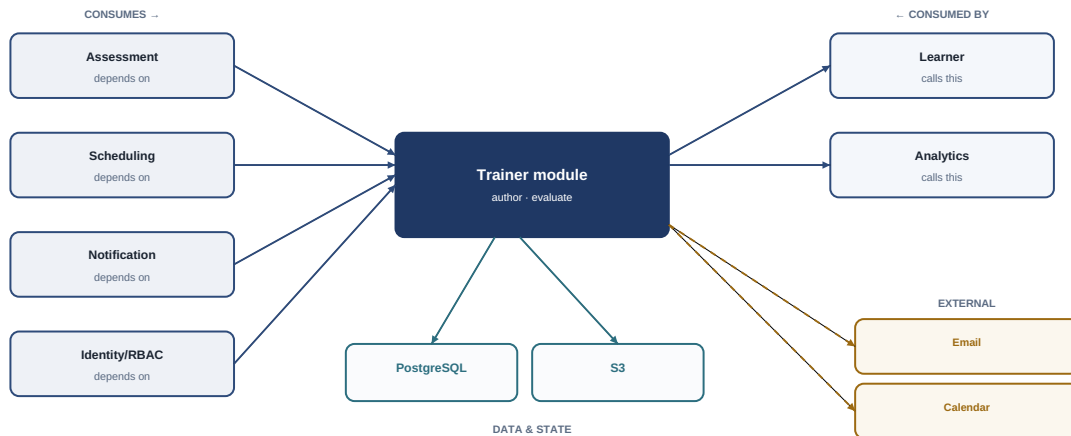


Figure M04-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Authoring Service	Course/module/submodule/content CRUD and versioning.
Content Media Service	Presigned upload/serve of content media via S3.
Evaluation Service	Trainer evaluations, AI-score override, learner feedback.
Schedule View	Read-only trainer schedule from Scheduling with reminders.
Task/Ticket Service	Trainer tasks and learner-raised tickets.

HLD — Interfaces

Provides

- Course/content read APIs (consumed by Learner).
- Evaluation results (consumed by Assessment/Analytics).
- Event: course-published, evaluation-recorded.

Consumes

- Assessment — author tests, override scores.
- Scheduling — read schedule, reminders.
- Notification — S3 presign, reminder mails.

- Identity/RBAC — auth, scopes.

HLD — Request & Data Flow

- Author course → **Authoring Service** writes course/module/submodule/content; media via presigned S3.
- Create assessment → **Assessment** authoring; assign to batch.
- Learner submits → **Assessment** AI-scores → trainer reviews in **Evaluation Service** and may override.
- View schedule → read confirmed bookings from **Scheduling**; reminders via Notification.
- Publish course → version frozen; **course-published** event emitted.

Scaling & performance. Performance: Content read < 300 ms (cacheable); media via CDN-fronted S3. **Scales by:** Stateless; content reads served from replicas/CDN.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

course

A course owned/authored by a trainer.

Field	Type	Notes
course_id	uuid	PK
trainer_id	uuid	FK, IDX
domain_id	uuid	FK
title	varchar(160)	
summary	text	
level	varchar(16)	BEGINNER INTERMEDIATE ADVANCED
version	int	incremented on publish
status	varchar(16)	DRAFT PUBLISHED ARCHIVED
created_at	timestampz	

course_module

A top-level section of a course.

Field	Type	Notes
module_id	uuid	PK
course_id	uuid	FK, IDX
title	varchar(160)	
sort_order	int	

submodule

A lesson within a module.

Field	Type	Notes
submodule_id	uuid	PK
module_id	uuid	FK, IDX
title	varchar(160)	

sort_order	int	
est_minutes	int	

content

A content block inside a submodule.

Field	Type	Notes
content_id	uuid	PK
submodule_id	uuid	FK, IDX
type	varchar(12)	TEXT CODE IMAGE PDF VIDEO
body	text	for TEXT/CODE
s3_key	varchar(255)	for media
language	varchar(24)	for CODE
sort_order	int	

trainer_evaluation

A trainer's evaluation of a learner submission.

Field	Type	Notes
evaluation_id	uuid	PK
trainer_id	uuid	FK
learner_id	uuid	FK
result_id	uuid	FK → test_result
override_score	numeric(5,2)	nullable
comments	text	
created_at	timestampz	

learner_feedback

Trainer → learner qualitative feedback.

Field	Type	Notes
feedback_id	uuid	PK
trainer_id	uuid	FK
learner_id	uuid	FK
batch_id	uuid	FK
notes	text	
created_at	timestampz	

task

A trainer task/to-do tied to delivery.

Field	Type	Notes
task_id	uuid	PK
trainer_id	uuid	FK, IDX
title	varchar(160)	
due_at	timestampz	nullable
status	varchar(16)	OPEN DONE

ticket

A learner-raised query to the trainer.

Field	Type	Notes
ticket_id	uuid	PK
learner_id	uuid	FK
trainer_id	uuid	FK
subject	varchar(160)	

body	text	
status	varchar(16)	OPEN ANSWERED CLOSED
created_at	timestampz	

LLD — Service Methods

AuthoringService

```
Course createCourse(CourseForm f)
```

Creates a DRAFT course for the trainer.

```
CourseModule addModule(CourseId c, String title, int order)
```

Adds a module to a course.

```
Submodule addSubmodule(ModuleId m, SubmoduleForm f)
```

Adds a lesson to a module.

```
Content addContent(SubmoduleId s, ContentForm f)
```

Adds a content block; media uploaded via presigned S3.

```
void publish(CourseId c)
```

Freezes the current version and emits course-published; future edits bump the version.

EvaluationService

```
TrainerEvaluation evaluate(ResultId r, BigDecimal score, String comments)
```

Records an evaluation; overrides the AI score as authoritative.

```
LearnerFeedback giveFeedback(LearnerId l, BatchId b, String notes)
```

Stores learner-level qualitative feedback.

ScheduleView

```
List<Booking> mySchedule(TrainerId t, DateRange r)
```

Reads confirmed bookings for the trainer from Scheduling (read-only).

TicketService

```
Ticket answer(TicketId id, String body)
```

Posts a trainer answer and moves the ticket to ANSWERED.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/trainer/courses	Create a course.	TRN : COURSE : MANAGE
POST /api/v1/trainer/courses/{id}/modules	Add a module.	TRN : COURSE : MANAGE
POST /api/v1/trainer/submodules/{id}/content	Add content.	TRN : COURSE : MANAGE
POST /api/v1/trainer/courses/{id}/publish	Publish a course version.	TRN : COURSE : MANAGE
POST /api/v1/trainer/evaluations	Record/override an evaluation.	TRN : EVAL : MANAGE
POST /api/v1/trainer/feedback	Give learner feedback.	TRN : EVAL : MANAGE
GET /api/v1/trainer/schedule	View confirmed schedule.	TRN : SCHEDULE : VIEW
POST /api/v1/trainer/tickets/{id}/answer	Answer a ticket.	TRN : TICKET : MANAGE

LLD — Key Logic & Algorithms

Content versioning

Published content is immutable; edits create a new version so learners always see a consistent snapshot.

- Authoring happens on a DRAFT version of the course.
- Publish freezes the version and emits course-published.
- Any later edit creates version N+1 in DRAFT, leaving N intact for in-flight learners.
- Learner progress references the version it started against.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Content read < 300 ms (cacheable); media via CDN-fronted S3.
Integrity	Published versions immutable; overrides audited.
Scalability	Stateless; content reads served from replicas/CDN.
Usability	Rich authoring with code/image/PDF/video blocks.

M05 — Learning Experience & Engagement (Learner)

The Learner module delivers the participant experience: consuming course content, attempting tests and quizzes (via the Assessment engine), tracking attendance and progress, earning badges and awards, raising queries, and seeing events. It owns progress/attendance/award data and consumes the content and assessment engines.

Owning team: Learner.

Scope

- Course player — navigate modules/submodules and content, track progress.
- Test/quiz attempts within an open window, with auto-lock on timeout.
- Attendance for live sessions.
- Badges, awards and certificates from performance.
- Tickets/queries to trainers and an event carousel.

HLD — Architecture & Context

The Learner module is a Next.js course player plus a lightweight Progress/Engagement service. It reads content from Trainer, attempts/results from Assessment, invites from Scheduling, and serves media via presigned S3. It owns progress, attendance, badges/awards and tickets. It comes online in Phase 2 against the now-real Assessment engine.

Figure M05-1 — Learner — Learning Experience: HLD Context

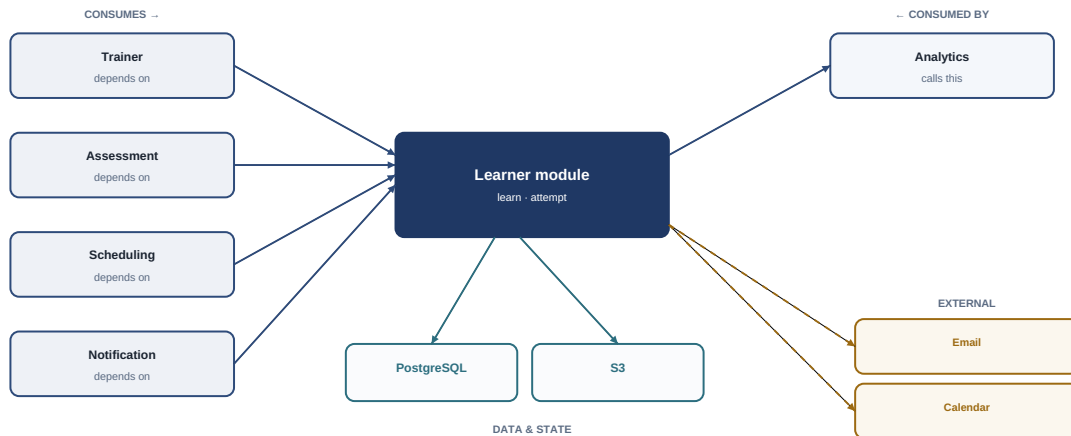


Figure M05-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Course Player (FE)	Renders content, tracks progress, resumes.
Progress Service	Lesson/course progress, completion, resume points.
Attempt Facade	Starts/submit attempts via the Assessment engine; surfaces results.
Attendance Service	Records attendance for live sessions.
Reward Service	Badges, awards and certificates from results.
Ticket Service	Learner queries to trainers.

HLD — Interfaces

Provides

- Engagement signals (progress, attendance) — consumed by Analytics.
- Event: lesson-completed, attempt-submitted.

Consumes

- Trainer — course/content read APIs.
- Assessment — start/submit attempts, results.
- Scheduling — session invites/reminders.

- Notification — S3 presign, reminders, event mails.

HLD — Request & Data Flow

- Open course → read content from **Trainer**; **Progress Service** records position.
- Start test → **Attempt Facade** → Assessment opens an attempt; submit before window close.
- Assessment AI-scores → learner views results/feedback once finalised.
- Attend live session → **Attendance Service** records presence.
- Earn badge/award → **Reward Service** derives from official results; certificate on completion.

Scaling & performance. **Performance:** Course player load < 400 ms; media via CDN. **Scales by:** Read-heavy; served from replicas/CDN.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

enrolment_progress

A learner's overall progress in a course.

Field	Type	Notes
progress_id	uuid	PK
learner_id	uuid	FK, IDX
course_id	uuid	FK
course_version	int	
percent_complete	numeric(5,2)	
last_submodule_id	uuid	resume point
status	varchar(16)	IN_PROGRESS COMPLETED
updated_at	timestampz	

lesson_progress

Per-submodule completion.

Field	Type	Notes
id	uuid	PK
learner_id	uuid	FK
submodule_id	uuid	FK
completed	boolean	
seconds_spent	int	
completed_at	timestampz	nullable

attempt_ref

Pointer to an Assessment attempt the learner started.

Field	Type	Notes
ref_id	uuid	PK
learner_id	uuid	FK, IDX

test_id	uuid	FK
attempt_id	uuid	Assessment id
status	varchar(16)	OPEN SUBMITTED LOCKED
started_at	timestampz	

attendance

Attendance for a live session.

Field	Type	Notes
attendance_id	uuid	PK
learner_id	uuid	FK, IDX
booking_id	uuid	FK → training_booking
status	varchar(12)	PRESENT ABSENT LATE
marked_at	timestampz	

badge

A badge a learner has earned.

Field	Type	Notes
badge_id	uuid	PK
learner_id	uuid	FK
code	varchar(48)	
label	varchar(120)	
awarded_at	timestampz	

award

A higher-order award (e.g. top performer).

Field	Type	Notes
award_id	uuid	PK
learner_id	uuid	FK
title	varchar(120)	
basis	varchar(160)	
awarded_at	timestampz	

certificate

A completion certificate.

Field	Type	Notes
certificate_id	uuid	PK
learner_id	uuid	FK
course_id	uuid	FK
s3_key	varchar(255)	PDF
issued_at	timestampz	

learner_ticket

A query raised by the learner.

Field	Type	Notes
ticket_id	uuid	PK
learner_id	uuid	FK
trainer_id	uuid	FK
subject	varchar(160)	
status	varchar(16)	OPEN ANSWERED CLOSED
created_at	timestampz	

LLD — Service Methods

ProgressService

```
List<Course> assignedCourses(LearnerId l)
```

Lists courses assigned to the learner's batches via course_assignment (set by the Organiser).

```
EnrolmentProgress openCourse(LearnerId l, CourseId c)
```

Loads or creates progress; returns the resume point.

```
void recordLesson(LearnerId l, SubmoduleId s, int seconds)
```

Marks a lesson complete and updates percent_complete.

```
void completeCourse(LearnerId l, CourseId c)
```

Marks completion and triggers certificate issuance.

AttemptFacade

```
Attempt start(LearnerId l, TestId t)
```

Opens an attempt via Assessment if the window is open and attempts remain.

```
Result submit(AttemptId a, Answers ans)
```

Submits answers; Assessment scores; returns the result once finalised.

AttendanceService

```
void mark(LearnerId l, BookingId b, Status s)
```

Records attendance for a live session.

RewardService

```
List<Badge> badges(LearnerId l)
```

Returns earned badges.

```
Certificate issueCertificate(LearnerId l, CourseId c)
```

Generates a completion certificate (PDF on S3).

LLD — API Endpoints

Method & path	Purpose	RBAC scope
GET /api/v1/learner/courses/{id}	Open a course and content.	LRN:COURSE:VIEW
POST /api/v1/learner/progress	Record lesson progress.	LRN:COURSE:VIEW
POST /api/v1/learner/tests/{id}/attempts	Start an attempt.	LRN:TEST:ATTEMPT
POST /api/v1/learner/attempts/{id}/submit	Submit an attempt.	LRN:TEST:ATTEMPT
GET /api/v1/learner/results	View results & feedback.	LRN:TEST:ATTEMPT
POST /api/v1/learner/attendance	Mark attendance.	LRN:ATTEND:CREATE
GET /api/v1/learner/badges	View badges/awards.	LRN:COURSE:VIEW
POST /api/v1/learner/tickets	Raise a ticket.	LRN:TICKET:CREATE

LLD — Key Logic & Algorithms

Attempt window enforcement

Attempts are valid only inside the open window and within the allowed count; closure auto-locks.

- On start, verify the test window is open and the learner has attempts remaining.
- Open an attempt in Assessment and track it via attempt_ref.

- On submit (or at window close / timeout) the attempt locks; no further edits.
- Assessment scores; once finalised (post any trainer override) the result and feedback are shown.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Course player load < 400 ms; media via CDN.
Reliability	Progress autosaved; attempts survive disconnect up to window close.
Scalability	Read-heavy; served from replicas/CDN.
Accessibility	Keyboard navigable, responsive player.

M06 — Scheduling & Conflict Resolution Engine

This engine guarantees that a trainer is never double-booked across any organisation, gates every booking behind Manager approval, and releases a slot the instant a training is held or cancelled. It owns the slot and booking model and exposes availability, propose, approve, hold and cancel operations to the modules that orchestrate them (Organiser, Manager, Trainer).

Owning team: Scheduling.

Scope

- Trainer slot model and availability lookup (time-zone-normalised).
- Distributed slot locking (Redlock) on the propose hot path.
- Cross-organisation conflict detection (overlap on UTC timestamps).
- Booking lifecycle state machine and Manager-gated transitions.
- Immediate slot release and real-time fan-out (pub/sub) on hold/cancel.

HLD — Architecture & Context

A Scheduling service backed by PostgreSQL (slots, bookings, approvals) and Redis (Redlock locks, pub/sub). The propose path takes a lock, runs an indexed overlap query, and writes the booking plus an outbox row in a single transaction. Approve/hold/cancel are state transitions that release locks and publish slot events; Notification drains the outbox for emails and calendar invites. The full design is detailed in the Organiser BRD/HLD/LLD.

Figure M06-1 — Scheduling & Conflict Engine: HLD Context

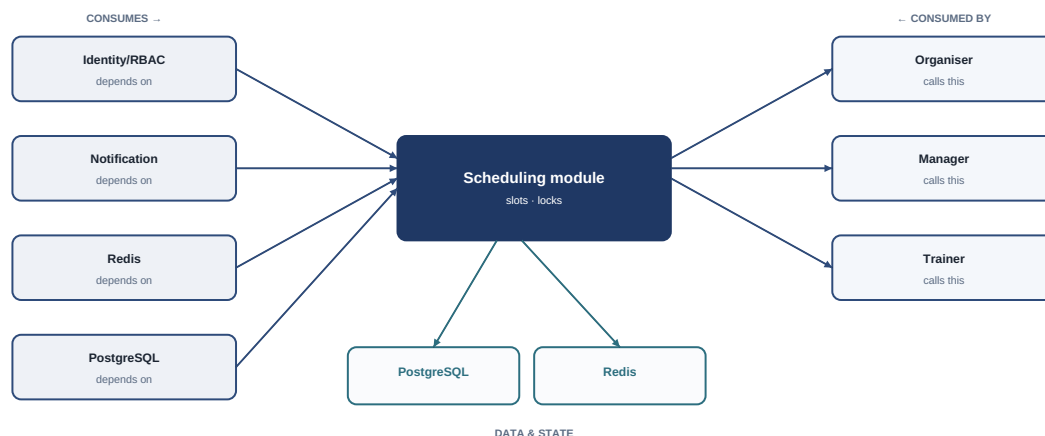


Figure M06-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Availability Service	Computes free slots for a trainer/range from slots minus active bookings.
Lock Manager	Redlock acquire/release keyed to trainer+window with fencing tokens.
Conflict Engine	Time-zone-normalised overlap detection across all organisations.
Booking Service	Booking creation, state machine, approval transitions, slot release.
Event Publisher	Redis pub/sub fan-out of slot-confirmed / slot-released.

HLD — Interfaces

Provides

- GET /trainers/{id}/availability.
- POST /trainings/propose | approve | hold | cancel.
- Events: slot-confirmed, slot-released (pub/sub + bus).

Consumes

- Identity/RBAC — auth/scopes; trainer ↔ org links (read).
- Notification — outbox enqueue for mails/calendar.

- Redis — locks and pub/sub; PostgreSQL — system of record.

HLD — Request & Data Flow

- Propose → acquire **Redis lock** on trainer+window.
- Run **Conflict Engine** overlap query against PostgreSQL (active bookings).
- If free → **Booking Service** INSERT booking (PENDING) + outbox row in one tx; release lock.
- Manager approve → transition CONFIRMED → **Event Publisher** slot-confirmed → outbox confirmations.
- Hold/cancel → release slot + publish slot-released → outbox notifications/calendar updates.

Scaling & performance. Performance: Availability p95 < 400 ms; propose/approve p95 < 1 s. **Scales by:** Stateless service; lock keeps the hot path race-free under load.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

training_slot

A bookable trainer time window; the unit of occupancy.

Field	Type	Notes
slot_id	uuid	PK
trainer_id	uuid	FK, IDX(trainer_id,start_ts,end_ts)
start_ts	timestamptz	UTC
end_ts	timestamptz	UTC
state	varchar(12)	OPEN RESERVED BOOKED
created_at	timestamptz	

training_booking

A proposed/confirmed training against a slot.

Field	Type	Notes
booking_id	uuid	PK
slot_id	uuid	FK
org_id	uuid	FK
batch_id	uuid	FK
organiser_id	uuid	FK → app_user
mode	varchar(12)	ONLINE ONSITE
venue	jsonb	{room,floor,location}
status	varchar(20)	DRAFT PENDING_APPROVAL CONFIRMED ON_HOLD REJECTED CANCELLED COMPLETED, IDX(status)
created_at	timestamptz	

booking_approval

The Manager's decision on a booking.

Field	Type	Notes
approval_id	uuid	PK

booking_id	uuid	FK, IDX
manager_id	uuid	FK → app_user
decision	varchar(12)	APPROVE REJECT HOLD
reason	varchar(255)	nullable
decided_at	timestampz	

trainer_org_link

Trainer ↔ org/domain link, read by conflict detection (custodian).

Field	Type	Notes
id	uuid	PK
trainer_id	uuid	FK, IDX
org_id	uuid	FK
domain_id	uuid	FK
active	boolean	

slot_lock_audit

Audit of lock acquire/release (for diagnostics).

Field	Type	Notes
id	bigserial	PK
trainer_id	uuid	IDX
window_key	varchar(80)	
token	uuid	fencing token
action	varchar(8)	ACQUIRE RELEASE EXPIRE
at	timestampz	

booking_transition

Immutable log of state transitions.

Field	Type	Notes
id	bigserial	PK
booking_id	uuid	FK, IDX
from_state	varchar(20)	
to_state	varchar(20)	
actor_id	uuid	FK
reason	varchar(255)	
at	timestampz	

reminder_schedule

Reminder cadence for a confirmed booking.

Field	Type	Notes
id	uuid	PK
booking_id	uuid	FK, IDX
fire_at	timestampz	IDX
kind	varchar(16)	DAILY H1 M30 M10
sent	boolean	

LLD — Service Methods

AvailabilityService

```
AvailabilityView availability(TrainerId t, DateRange r)
```

Returns free windows = slots minus active bookings, in the caller's tz.

```
List<Slot> suggestFree(TrainerId t, Window w, int n)
```

Returns the next N free windows near a requested time (for 409 responses).

SchedulingService

```
ProposalResult propose(ProposeCommand cmd)
```

Acquires the Redlock, runs conflict detection, creates a PENDING booking + outbox row atomically; or returns 409 + suggestions.

```
void approve(BookingId id, ManagerId m)
```

Transitions to CONFIRMED, publishes slot-confirmed, enqueues confirmations + reminders.

```
void hold(BookingId id, ManagerId m, String reason)
```

Transitions to ON_HOLD, releases the slot immediately, publishes slot-released.

```
void cancel(BookingId id, ManagerId m, String reason)
```

Transitions CONFIRMED → CANCELLED, frees the slot, notifies all parties.

```
void complete(BookingId id)
```

Marks COMPLETED after the session; feeds analytics.

ConflictEngine

```
ConflictResult detect(TrainerId t, Window w)
```

Runs the UTC overlap query across all orgs; returns FREE or the overlapping booking ids.

LockManager

```
Lock acquire(String key, Duration ttl)
```

SET key token NX PX ttl; returns a fencing-token lock or null.

```
void release(Lock lock)
```

Compare-and-delete via Lua; only the owner releases.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
GET /api/v1/trainers/{id}/availability	Free slots for a range.	SCHEDULING:VIEW
POST /api/v1/trainings/propose	Create a conflict-safe proposal.	SCHEDULING:CREATE
POST /api/v1/trainings/{id}/approve	Confirm (Manager).	MGR:APPROVAL:DECIDE
POST /api/v1/trainings/{id}/hold	Hold + release slot (Manager).	MGR:APPROVAL:DECIDE
POST /api/v1/trainings/{id}/cancel	Cancel + free slot (Manager).	MGR:APPROVAL:DECIDE
POST /api/v1/trainings/{id}/complete	Mark completed.	SCHEDULING:UPDATE

LLD — Key Logic & Algorithms

Redlock slot lock

Serialises concurrent proposals for the same trainer window without DB row-locking.

```
key = "lock:trainer:" + trainerId + ":" + windowBucket(start,end)
token = uuid()
ok = SET key token NX PX 30000 # 30s TTL
if not ok: -> 423 LOCKED (client retries)
# ... conflict detect + insert booking (+ outbox) ...
if redis.get(key) == token: redis.del(key) # safe release (Lua CAS)
```

Cross-org overlap query

Existence check on the half-open interval, filtered by trainer only.

```
SELECT b.booking_id
FROM training_booking b JOIN training_slot s ON s.slot_id=b.slot_id
WHERE s.trainer_id = :trainerId
      AND b.status IN ('PENDING_APPROVAL','CONFIRMED')
      AND s.start_ts < :endTs AND :startTs < s.end_ts
LIMIT 1;  -- row => OCCUPIED, else FREE
```

LLD — State / Lifecycle

From → To	Trigger	Effect
DRAFT → PENDING_APPROVAL	Organiser submits	Redis slot lock; approval-request via outbox.
PENDING → CONFIRMED	Manager approves	Durable booking; invites + reminders; publish slot-confirmed.
PENDING → REJECTED	Manager rejects	Lock released; slot freed; submitter notified.
* → ON_HOLD	Manager holds	Slot released immediately; all parties mailed.
CONFIRMED → CANCELLED	Manager cancels	Slot freed; calendars updated.
CONFIRMED → COMPLETED	Session delivered	Feeds analytics, badges, awards.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Availability p95 < 400 ms; propose/approve p95 < 1 s.
Consistency	Zero double-booking (lock + unique index).
Scalability	Stateless service; lock keeps the hot path race-free under load.
Resilience	Lock TTL auto-recovers from node crashes; replicas for reads.

M07 — Assessment & AI Evaluation Engine

This engine owns the full assessment lifecycle: authoring tests and questions (MCQ and theoretical), assigning them to learners/batches, running timed attempts with auto-lock, scoring with AI (with trainer override as authoritative), and exposing results and per-domain performance to the modules that need them.

Owning team: Assessment.

Scope

- Test and question authoring (MCQ, theoretical; text/code/image/PDF answers).
- Assignment to a learner, batch, university or organisation.
- Timed attempts with auto-lock on close/timeout.
- AI scoring with per-domain breakdown and feedback; trainer override.
- Results and performance retrieval per learner/batch.

HLD — Architecture & Context

An Assessment service backed by PostgreSQL (tests, questions, attempts, results) with an AI adapter for scoring and S3 for media. Authoring and assignment are synchronous; scoring runs on submit (sync or via a worker for heavy items); override updates the official result. Results feed Analytics and the Learner reward flow. Built in Phase 1 with a stub scorer, AI wired progressively.

Figure M07-1 — Assessment & AI Evaluation: HLD Context

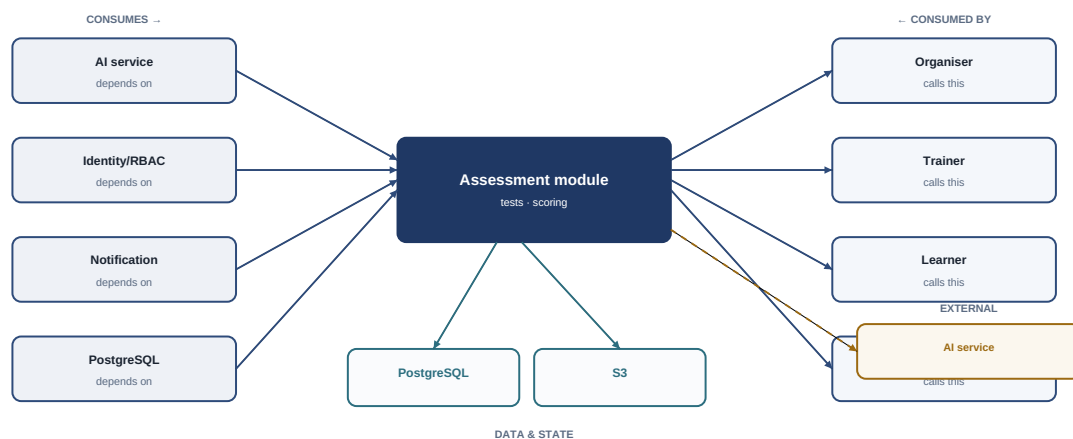


Figure M07-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Authoring Service	Test/question CRUD; assignment to learner/batch/org.
Attempt Service	Opens/closes attempts; enforces window and attempt limits.
Scoring Service	AI scoring with per-domain breakdown; calls the AI adapter.
Override Service	Trainer override of AI score/feedback; sets official result.
Results Service	Result and performance retrieval per learner/batch.

HLD — Interfaces

Provides

- POST /tests, /tests/{id}/assign, /attempts; GET /tests/{id}/results.
- Event: test-graded.

Consumes

- AI service — question-gen/scoring.
- Identity/RBAC — auth/scopes; Notification — S3 presign, result mails.
- PostgreSQL — system of record; S3 — media answers.

HLD — Request & Data Flow

- Author test → **Authoring Service** writes test/questions → assign to batch.
- Learner attempt → **Attempt Service** opens an attempt within the window.
- Submit → **Scoring Service** AI-scores → result PENDING_REVIEW.
- Trainer override (optional) → **Override Service** sets official score.
- Finalise → emit **test-graded** → Analytics + Learner rewards.

Scaling & performance. **Performance:** Authoring/assign < 500 ms; MCQ scoring near-instant; theory async. **Scales by:** Scoring offloaded to workers for heavy items.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

test

An assessment definition.

Field	Type	Notes
test_id	uuid	PK
created_by	uuid	FK → app_user
domain_id	uuid	FK
title	varchar(160)	
type	varchar(16)	MCQ THEORETICAL MIXED
min_marks	int	
max_marks	int	
duration_min	int	
attempts_allowed	int	
opens_at	timestampz	
closes_at	timestampz	
status	varchar(16)	DRAFT PUBLISHED CLOSED

question

A question within a test.

Field	Type	Notes
question_id	uuid	PK
test_id	uuid	FK, IDX
kind	varchar(16)	MCQ THEORY
prompt	text	
answer_type	varchar(12)	TEXT CODE IMAGE PDF
marks	int	
options	jsonb	for MCQ
answer_key	jsonb	nullable
domain_tag	varchar(48)	

assignment

A test assigned to a target.

Field	Type	Notes
assignment_id	uuid	PK
test_id	uuid	FK
target_type	varchar(16)	LEARNER BATCH ORG UNIVERSITY
target_id	uuid	
assigned_by	uuid	FK
assigned_at	timestampz	

attempt

A learner's attempt at a test.

Field	Type	Notes
attempt_id	uuid	PK
test_id	uuid	FK, IDX
learner_id	uuid	FK, IDX
started_at	timestampz	
submitted_at	timestampz	nullable
status	varchar(16)	OPEN SUBMITTED LOCKED GRADED
attempt_no	int	

answer

A learner's answer to a question.

Field	Type	Notes
answer_id	uuid	PK
attempt_id	uuid	FK, IDX
question_id	uuid	FK
response	text	TEXT/CODE
s3_key	varchar(255)	IMAGE/PDF
auto_score	numeric(5,2)	nullable

test_result

The official result for an attempt.

Field	Type	Notes
result_id	uuid	PK
attempt_id	uuid	FK, UNIQUE
learner_id	uuid	FK, IDX
ai_score	numeric(5,2)	
official_score	numeric(5,2)	= override or ai
domain_breakdown	jsonb	per-domain scores
ai_feedback	text	
override_by	uuid	FK, nullable
finalised_at	timestampz	nullable

score_override

A trainer override of an AI score.

Field	Type	Notes
override_id	uuid	PK
result_id	uuid	FK
trainer_id	uuid	FK
new_score	numeric(5,2)	

reason	varchar(255)	
at	timestampz	

LLD — Service Methods

AuthoringService

```
Test createTest(TestForm f)
```

Creates a test with timing, marks and questions.

```
Question addQuestion(TestId t, QuestionForm q)
```

Adds an MCQ/theory question with answer type and domain tag.

```
Assignment assign(TestId t, Target target, UserId by)
```

Assigns the test to a learner/batch/org/university.

AttemptService

```
Attempt open(LearnerId l, TestId t)
```

Opens an attempt if within window and attempts remain; else rejects.

```
Result submit(AattemptId a, List<Answer> answers)
```

Persists answers, locks the attempt, triggers scoring.

```
void autoLock(AattemptId a)
```

Locks an attempt at window close/timeout.

ScoringService

```
ScoreResult score(AattemptId a)
```

Calls the AI adapter; produces per-domain scores and feedback; writes a PENDING result.

OverrideService

```
void override(ResultId r, TrainerId t, BigDecimal score, String reason)
```

Sets the official score to the trainer's value and finalises.

ResultsService

```
ResultsView byLearner(LearnerId l, Filter f)
```

Results and performance for a learner.

```
ResultsView byBatch(BatchId b, Filter f)
```

Aggregated results for a batch.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/tests	Create a test.	ASSESS:CREATE
POST /api/v1/tests/{id}/questions	Add a question.	ASSESS:CREATE
POST /api/v1/tests/{id}/assign	Assign the test.	ASSESS:CREATE
POST /api/v1/tests/{id}/attempts	Open an attempt.	LRN:TEST:ATTEMPT
POST /api/v1/attempts/{id}/submit	Submit an attempt.	LRN:TEST:ATTEMPT
POST /api/v1/results/{id}/override	Override the AI score.	TRN:EVAL:MANAGE

LLD — Key Logic & Algorithms

Submit → score → finalise

Scoring is assistive; finalisation respects any trainer override.

- On submit, lock the attempt and persist answers.
- ScoringService calls the AI adapter → per-domain scores + feedback → result status PENDING_REVIEW.
- If a trainer overrides, official_score = override; else official_score = ai_score after a review window.
- On finalisation, set finalised_at and emit test-graded for analytics and rewards.

LLD — State / Lifecycle

From → To	Trigger	Effect
OPEN → SUBMITTED	Learner submits	Answers persisted; scoring triggered.
OPEN → LOCKED	Window close / timeout	Attempt auto-locked; scored as-is.
SUBMITTED → GRADED	Scoring + finalisation	Official result set; test-graded emitted.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Authoring/assign < 500 ms; MCQ scoring near-instant; theory async.
Integrity	Window/attempt limits enforced; overrides audited.
Scalability	Scoring offloaded to workers for heavy items.
Accuracy	Human override authoritative over AI.

M08 — Notification & Integration Services

This module delivers every side-effect in the platform reliably and asynchronously: transactional emails, Google Calendar invites/updates, reminder cadences, and presigned S3 media access. Producers enqueue work via a transactional outbox; workers deliver it with retries and dead-lettering, keeping latency and third-party failure off the request path.

Owning team: Notifications.

Scope

- Transactional outbox + worker pool for exactly-once-ish side-effects.
- Email/SMTP delivery with templates and attachments.
- Google Calendar invite/update and reminder scheduling.
- S3 presigned URL issuance for media (5-min TTL, no public ACL).
- Retry, back-off and dead-letter handling.

HLD — Architecture & Context

A Notification service with a relay worker that polls/streams the outbox table, plus adapters for email, calendar and S3. Producers write an outbox row in their own transaction; the relay delivers with retry/back-off and dead-letters poison messages. Reminder schedules are stored and fired by a scheduler. Built in Phase 1 with an email stub; providers wired progressively.

Figure M08-1 — Notification & Integration: HLD Context

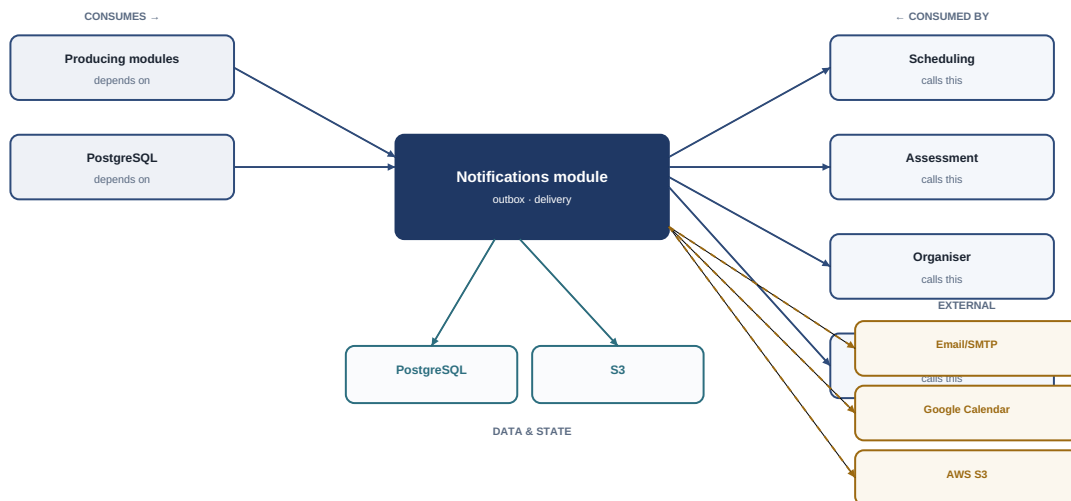


Figure M08-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Outbox Relay	Drains committed outbox rows; orchestrates delivery + retry/DLQ.
Email Adapter	Templated SMTP delivery with attachments.
Calendar Adapter	Idempotent Google Calendar invite/update; reminders.
Media/Presign Service	Issues short-lived presigned S3 URLs.
Reminder Scheduler	Fires reminder cadences; cancels on booking change.

HLD — Interfaces

Provides

- POST /outbox (enqueue), POST /media/presign.
- Email/calendar payload schemas; delivery status API.

Consumes

- Email/SMTP, Google Calendar API, AWS S3 — external providers.
- Producing modules' outbox rows; PostgreSQL — outbox/log.

HLD — Request & Data Flow

- Producer writes an **outbox row** in its own transaction (commit = durable intent).
- **Outbox Relay** picks up committed rows and routes to the right adapter.
- **Email/Calendar adapters** deliver idempotently; success/failure recorded.
- On failure → retry with back-off → **dead-letter** after N attempts.
- **Reminder Scheduler** fires cadences and cancels on booking change.

Scaling & performance. Performance: Enqueue is a single insert; delivery is off the request path. **Scales by:** Workers scale horizontally on queue depth.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

outbox

A durable side-effect intent committed with the producer's data.

Field	Type	Notes
outbox_id	uuid	PK
aggregate_type	varchar(48)	e.g. BOOKING
aggregate_id	uuid	
type	varchar(48)	EMAIL CALENDAR REMINDER
payload	jsonb	
idempotency_key	varchar(120)	UNIQUE
status	varchar(16)	PENDING SENT FAILED DEAD, IDX
attempts	int	
next_attempt_at	timestamptz	IDX
created_at	timestamptz	

notification_log

Record of attempted/sent notifications.

Field	Type	Notes
log_id	bigserial	PK
outbox_id	uuid	FK
channel	varchar(12)	EMAIL CALENDAR
recipient	varchar(255)	
provider_msg_id	varchar(160)	nullable
result	varchar(12)	SENT FAILED
error	varchar(255)	nullable
at	timestamptz	

email_template

A named, versioned email template.

Field	Type	Notes
template_id	uuid	PK

key	varchar(64)	UNIQUE, e.g. BOOKING_CONFIRMED
subject	varchar(200)	
body_html	text	
version	int	

calendar_event_link

Mapping of a booking to its external calendar event.

Field	Type	Notes
id	uuid	PK
booking_id	uuid	FK, UNIQUE
provider	varchar(16)	GOOGLE
external_event_id	varchar(200)	
status	varchar(12)	ACTIVE CANCELLED
updated_at	timestampz	

reminder_job

A scheduled reminder to fire.

Field	Type	Notes
job_id	uuid	PK
booking_id	uuid	FK, IDX
fire_at	timestampz	IDX
kind	varchar(16)	DAILY H1 M30 M10
sent	boolean	
cancelled	boolean	

dead_letter

Side-effects that failed past the retry limit.

Field	Type	Notes
id	uuid	PK
outbox_id	uuid	FK
payload	jsonb	
last_error	varchar(255)	
created_at	timestampz	

presign_audit

Audit of issued presigned URLs.

Field	Type	Notes
id	bigserial	PK
s3_key	varchar(255)	
issued_to	uuid	FK → app_user
ttl_seconds	int	
at	timestampz	

LLD — Service Methods

OutboxRelay

```
void enqueue(OutboxRow row)
```

Inserts an outbox row (called within the producer's transaction).

```
void drain()
```

Polls committed PENDING rows and dispatches to adapters; updates status.

```
void retryOrDeadLetter(OutboxRow row, Exception e)
```

Schedules a back-off retry or moves to `dead_letter` after N attempts.

EmailAdapter

```
DeliveryResult send(String templateKey, Recipient to, Map data, List<Attachment> files)
```

Renders the template and sends via SMTP; idempotent on the idempotency key.

CalendarAdapter

```
void upsertInvite(BookingId b, InviteSpec s)
```

Creates or updates the Google Calendar event idempotently by booking id.

```
void cancelInvite(BookingId b)
```

Cancels the external event on hold/cancel.

ReminderScheduler

```
void schedule(BookingId b, Cadence c)
```

Creates reminder jobs (daily, 1 hr/30/10 min).

```
void cancelAll(BookingId b)
```

Cancels pending reminders when the booking changes.

PresignService

```
String presignGet(String s3Key, Duration ttl)
```

Issues a short-lived presigned GET URL; audits the issuance.

```
String presignPut(String s3Key, Duration ttl)
```

Issues a presigned PUT URL for uploads.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/outbox	Enqueue a side-effect (internal).	service
POST /api/v1/media/presign	Issue a presigned S3 URL.	authenticated
GET /api/v1/notifications/{id}/status	Delivery status.	service
POST /api/v1/reminders/cancel	Cancel reminders for a booking.	service

LLD — Key Logic & Algorithms

Transactional outbox delivery

Guarantees a side-effect is never lost even if a node crashes right after commit.

- Producer writes business row + outbox row in ONE transaction.
- Relay reads committed PENDING rows (the commit is the durable signal).
- Adapter delivers idempotently; on success mark SENT, on failure schedule a back-off retry.
- After N failures move to `dead_letter` for inspection; never block the pipeline.

LLD — State / Lifecycle

From → To	Trigger	Effect
PENDING → SENT	Successful delivery	notification_log SENT.
PENDING → FAILED → PENDING	Transient error + back-off	Retry scheduled at next_attempt_at.
FAILED → DEAD	Retry limit reached	Moved to dead_letter.

LLD — Non-Functional Targets

Attribute	Target / approach
Reliability	No lost side-effects; idempotent delivery; DLQ for poison messages.
Performance	Enqueue is a single insert; delivery is off the request path.
Scalability	Workers scale horizontally on queue depth.
Security	Presigned URLs short-lived, IAM-scoped, never public.

M09 — Frontend Experience Platform & Design System

This module owns the shared frontend foundation: the Next.js App Router shell, the dynamic navigation driven by the user's resolved permissions, the design-token system (flat editorial, system fonts), and the component library every role module composes. It owns no business/domain data — its 'entities' are platform configuration (themes, tokens, navigation, feature flags) served to the role UIs.

Owning team: Frontend.

Scope

- Next.js App Router shell — routing, layouts, auth/session wiring, error boundaries.
- RBAC-driven dynamic navigation built from GET /me/permissions.
- Shared component library (tables, forms, modals, dashboards) and design tokens.
- Theme + token system: flat editorial, system fonts, no glassmorphism/gradient text.
- Feature flags and layout templates consumed by role modules.

HLD — Architecture & Context

A Next.js 14 App Router application providing the authenticated shell and a published component/token library (consumed as a package by role modules). It calls Identity/RBAC for permissions to build the navigation tree and route guards, and stores its own platform configuration (themes, tokens, nav metadata, flags) as JSON/static config or in a thin config service. It is a Phase-0 module so role teams build on it from the start.

Figure M09-1 — Frontend Platform & Design System: HLD Context

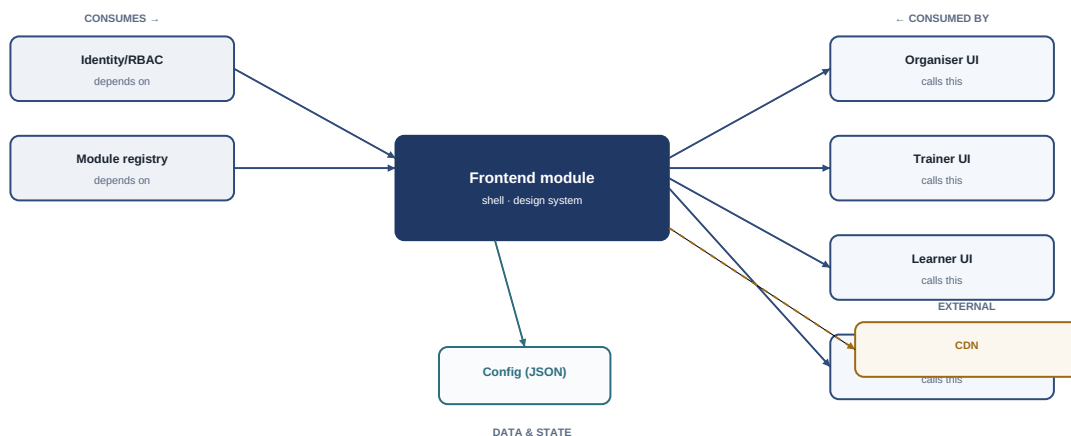


Figure M09-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
App Shell	Routing, layouts, auth/session, error boundaries, providers.
Dynamic Nav	Builds the menu/route tree from /me/permissions; hides unauthorised items.
Component Library	Shared, versioned UI components consumed by all role modules.
Design Tokens	Central colour/spacing/typography tokens (flat editorial).
Feature Flag Client	Evaluates flags to toggle UI without redeploy.

HLD — Interfaces

Provides

- The authenticated shell and routing conventions.
- Component library + design tokens (imported as a package).
- Dynamic-nav and route-guard utilities.

Consumes

- Identity/RBAC — GET /me/permissions for nav and guards; auth endpoints.

- Module registry (from RBAC) — module/route metadata.
- All module APIs — at the screen level (composed by role teams).

HLD — Request & Data Flow

- User logs in → shell stores the session → calls **GET /me/permissions**.
- **Dynamic Nav** builds the menu/route tree from the returned modules/scopes.
- Role screens **import the component library + tokens** and call their module APIs.
- Feature flags evaluated client-side to toggle capabilities.
- On permission change, next navigation re-fetches and re-renders nav.

Scaling & performance. Performance: First meaningful paint < 1.5 s; nav build < 50 ms client-side.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 App Router, JavaScript, CSS variables (design tokens)
Components	Shared library (versioned package), system-font stack
Auth	JWT in HttpOnly cookie; client session + refresh
Config	JSON/static tokens + thin config service
Delivery	Docker, GitHub Actions, CDN-fronted static assets

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

theme

A named visual theme (config, not domain data).

Field	Type	Notes
theme_id	uuid	PK
key	varchar(48)	UNIQUE, e.g. EDITORIAL
label	varchar(80)	
is_default	boolean	
updated_at	timestamptz	

design_token

A single design token belonging to a theme.

Field	Type	Notes
token_id	uuid	PK
theme_id	uuid	FK, IDX
category	varchar(24)	COLOR SPACING TYPOGRAPHY RADIUS
name	varchar(64)	e.g. --brand
value	varchar(64)	e.g. #1F3864
description	varchar(160)	

nav_item

Navigation metadata mapped to a module/route.

Field	Type	Notes
nav_id	uuid	PK
module_key	varchar(64)	matches RBAC module
label	varchar(80)	
icon	varchar(48)	
route	varchar(160)	
parent_id	uuid	nullable, for nesting
sort_order	int	

required_scope	varchar(96)	shown only if held
----------------	-------------	--------------------

feature_flag

A UI feature toggle.

Field	Type	Notes
flag_id	uuid	PK
key	varchar(64)	UNIQUE
enabled	boolean	
rollout_pct	int	0–100
audience	jsonb	role/org targeting
updated_at	timestampz	

layout_template

A reusable page layout.

Field	Type	Notes
layout_id	uuid	PK
key	varchar(48)	e.g. TWO_COLUMN
label	varchar(80)	
regions	jsonb	named slots
version	int	

component_doc

Published documentation for a library component.

Field	Type	Notes
doc_id	uuid	PK
component	varchar(64)	e.g. DataTable
version	varchar(16)	
props_schema	jsonb	
status	varchar(16)	STABLE BETA DEPRECATED

LLD — Service Methods

NavigationBuilder

```
NavTree buildNav(Permissions perms, List<NavItem> items)
```

Filters nav metadata to the user's scopes and assembles a nested menu/route tree.

```
boolean canRoute(Permissions perms, String route)
```

Client-side guard; returns whether the route's required scope is held.

ThemeService

```
Map<String,String> resolveTokens(ThemeKey key)
```

Returns the CSS-variable map for a theme (flat editorial default).

```
Theme getDefaultTheme()
```

Returns the platform default theme.

FeatureFlagClient

```
boolean isEnabled(String key, UserContext ctx)
```

Evaluates a flag against rollout % and audience targeting.

ComponentRegistry

```
ComponentDoc describe(String component, String version)
```

Returns the props schema and status for a library component.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
GET /api/v1/fe/nav	Nav metadata (filtered server-side by scope).	authenticated
GET /api/v1/fe/theme	Default theme tokens.	authenticated
GET /api/v1/fe/flags	Active feature flags for the user.	authenticated
GET /api/v1/fe/components	Component documentation index.	developer

LLD — Key Logic & Algorithms

Permission-driven navigation

The menu and routes are a pure function of the user's resolved scopes — never hardcoded per role.

- On load, fetch GET /me/permissions (modules + MODULE:ACTION scopes).
- Filter nav_item metadata to items whose required_scope the user holds.
- Assemble the nested menu/route tree and render; unauthorised items never appear.
- Client guards block direct navigation; the server still enforces on every API call.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	First meaningful paint < 1.5 s; nav build < 50 ms client-side.
Consistency	Single token source; no per-module styling drift.
Accessibility	WCAG AA; keyboard navigable; responsive.
Maintainability	Versioned component library; documented props.

M10 — Analytics, Reporting & Insights

This module turns the events and data the platform produces into insight. It consumes domain events (test-graded, slot-confirmed, attendance, feedback) and read replicas, maintains read-only materialised aggregates, and serves operational and leadership dashboards plus scheduled/exported reports. It owns only derived, read-only data — never a system of record.

Owning team: Data.

Scope

- Event ingestion from all modules (test-graded, slot-confirmed, attendance, feedback).
- Materialised aggregates (per trainer, batch, learner, organisation, domain).
- Operational and leadership dashboards with widgets.
- Scheduled reports and on-demand exports (CSV/PDF).
- Report definitions and metric snapshots over time.

HLD — Architecture & Context

A Data-squad service with an ingestion worker consuming the event bus, an aggregate store (materialised views / summary tables on replicas), and a query/dashboard API. Reports are defined declaratively and run against aggregates; scheduled reports enqueue Notification mails. It is a Phase-3 module that becomes meaningful once Phase-1 data exists, but its event/read contracts are defined in Phase 1.

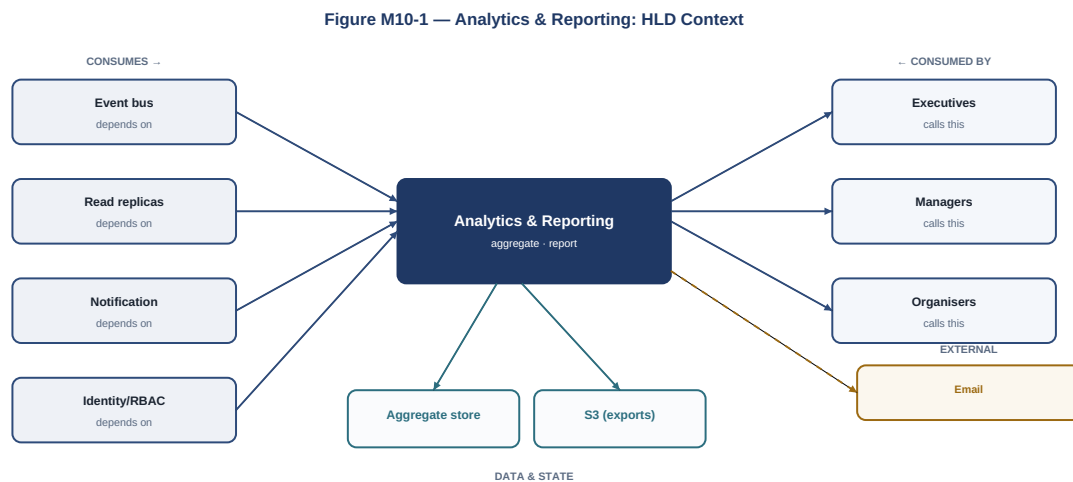


Figure M10-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Ingestion Worker	Consumes domain events and lands them in the analytics store.
Aggregation Engine	Builds/refreshes materialised aggregates; supports replay.
Dashboard Service	Serves scoped dashboards and widgets.
Report Engine	Runs parameterised report definitions; exports CSV/PDF.
Scheduler	Triggers scheduled reports and enqueues delivery.

HLD — Interfaces

Provides

- Dashboard and report query APIs.
- Scheduled report + export jobs.

Consumes

- Event bus — test-graded, slot-confirmed, attendance, feedback, booking events.
- Read replicas — source tables for aggregation.
- Notification — scheduled-report emails; Identity/RBAC — scoping.

HLD — Request & Data Flow

- Modules emit domain events → **Ingestion Worker** lands them.
- **Aggregation Engine** refreshes materialised aggregates (incremental or replay).
- Viewer opens a dashboard → **Dashboard Service** reads scoped aggregates.
- Scheduled report fires → **Report Engine** runs the definition → **Scheduler** enqueues a Notification email.
- Export requested → Report Engine generates CSV/PDF (PDF to S3).

Scaling & performance. Performance: Dashboard read < 500 ms from aggregates; no OLTP impact. **Scales by:** Ingestion scales on event volume; reads from replicas.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

report_definition

A reusable, parameterised report spec.

Field	Type	Notes
definition_id	uuid	PK
key	varchar(64)	UNIQUE
title	varchar(160)	
query_spec	jsonb	dimensions/metrics/filters
default_params	jsonb	
scope	varchar(16)	GLOBAL ORG BATCH
created_by	uuid	FK

materialised_aggregate

A precomputed summary row (derived, rebuildable).

Field	Type	Notes
agg_id	uuid	PK
grain	varchar(24)	TRAINER BATCH LEARNER ORG DOMAIN, IDX
grain_id	uuid	IDX
period	varchar(16)	DAY WEEK MONTH
metrics	jsonb	{avg_score,attendance_pct,...}
refreshed_at	timestampz	

metric_snapshot

An immutable point-in-time metric value (for trends).

Field	Type	Notes
snapshot_id	bigserial	PK
metric_key	varchar(64)	IDX
grain_id	uuid	IDX
value	numeric(14,4)	
captured_at	timestampz	IDX

dashboard

A named dashboard.

Field	Type	Notes
dashboard_id	uuid	PK
key	varchar(64)	UNIQUE
title	varchar(160)	
scope	varchar(16)	GLOBAL ORG ROLE
owner_role	varchar(48)	
created_at	timestamptz	

dashboard_widget

A widget on a dashboard.

Field	Type	Notes
widget_id	uuid	PK
dashboard_id	uuid	FK, IDX
type	varchar(16)	KPI TREND BAR TABLE
definition_id	uuid	FK → report_definition
position	jsonb	{x,y,w,h}
title	varchar(120)	

scheduled_report

A report scheduled to run and deliver.

Field	Type	Notes
schedule_id	uuid	PK
definition_id	uuid	FK
cron	varchar(40)	
recipients	jsonb	emails
format	varchar(8)	CSV PDF
active	boolean	
next_run_at	timestamptz	IDX

export_job

An on-demand export job.

Field	Type	Notes
job_id	uuid	PK
definition_id	uuid	FK
params	jsonb	
status	varchar(16)	QUEUED RUNNING DONE FAILED
s3_key	varchar(255)	nullable
requested_by	uuid	FK
created_at	timestamptz	

LLD — Service Methods

IngestionWorker

```
void onEvent(DomainEvent e)
```

Lands a domain event into the analytics store and marks aggregates dirty.

AggregationEngine

```
void refresh(Grain grain, Period period)
```

Recomputes the affected materialised aggregate (incremental).

```
void rebuild(Grain grain)
```

Replays source/events to rebuild an aggregate from scratch.

```
void snapshot(String metricKey)
```

Captures an immutable point-in-time metric value for trends.

DashboardService

```
DashboardView render(DashboardKey k, Viewer v)
```

Returns widgets resolved against scoped aggregates for the viewer.

ReportEngine

```
ReportResult run(DefinitionId d, Params p, Viewer v)
```

Runs a parameterised report scoped to the viewer.

```
ExportJob export(DefinitionId d, Params p, Format fmt)
```

Generates a CSV/PDF export (PDF to S3).

ReportScheduler

```
void fireDue()
```

Runs scheduled reports whose next_run_at has passed and enqueues delivery.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
GET /api/v1/reports/dashboards/{key}	Render a scoped dashboard.	RPT:DASHBOARD:VIEW
POST /api/v1/reports/{defId}/run	Run a report.	RPT:VIEW
POST /api/v1/reports/{defId}/export	Export a report (CSV/PDF).	RPT:EXPORT
POST /api/v1/reports/schedules	Create a scheduled report.	RPT:SCHEDULE:MANAGE
GET /api/v1/reports/exports/{jobId}	Export job status / download.	RPT:EXPORT

LLD — Key Logic & Algorithms

Event ingestion → incremental aggregation

Aggregates stay fresh without scanning the OLTP primary, and can be rebuilt from source for correctness.

- Consume a domain event (e.g. test-graded) and append it to the analytics store.
- Mark the affected aggregate grain (trainer/batch/learner/org) dirty.
- AggregationEngine incrementally recomputes the dirty aggregate on a schedule.
- Dashboards read the materialised aggregate; a periodic snapshot records trends.
- If integrity is questioned, rebuild() replays source/events to regenerate the aggregate.

LLD — Non-Functional Targets

Attribute	Target / approach
Performance	Dashboard read < 500 ms from aggregates; no OLTP impact.
Correctness	Aggregates rebuildable from source (replayable).
Scalability	Ingestion scales on event volume; reads from replicas.
Security	Every view scoped to the viewer's role/org.

M11 — Finance, Billing & Revenue

This module manages the commercial side of the platform: it prices and invoices confirmed training delivery against organisation contracts and billing rules, captures payments, and reconciles received funds against invoices. It consumes organisation/contract master data and booking-confirmed events, and owns the invoice, payment, contract and reconciliation domain.

Owning team: Finance.

Scope

- Contracts and billing rules per organisation (rate cards, taxes, currency).
- Invoice generation from confirmed/completed bookings and billing rules.
- Payment capture and recording.
- Reconciliation of payments against invoices.
- Credit notes and adjustments.

HLD — Architecture & Context

A Finance-squad service backed by PostgreSQL (contracts, invoices, payments, reconciliations). A worker consumes booking-confirmed/completed events and accrues billable items; invoicing applies billing rules + tax to produce invoices (PDF via Notification). Payments are recorded and reconciled against invoices, with credit notes for corrections. Built in Phase 2 once bookings are real.

Figure M11-1 — Finance & Billing: HLD Context

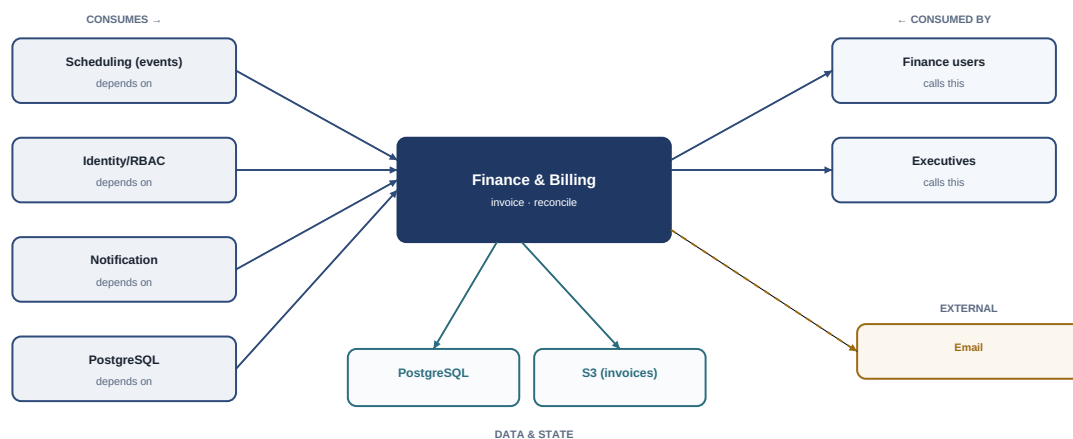


Figure M11-1 — HLD context: dependencies (left), consumers (right), data & external systems.

HLD — Component Responsibilities

Component	Responsibility
Contract Service	Organisation contracts, rate cards, billing rules, tax/currency.
Accrual Worker	Consumes booking events; creates billable items.
Invoicing Service	Generates invoices from accruals + rules; renders PDF.
Payment Service	Records full/partial payments against invoices.
Reconciliation Service	Matches payments to invoices; flags mismatches; credit notes.

HLD — Interfaces

Provides

- Invoice/payment/contract APIs; revenue & aging reports.
- Event: invoice-issued, payment-recorded.

Consumes

- Scheduling — booking-confirmed/completed events.
- Identity/RBAC — organisation/contract master data, scopes.
- Notification — invoice emails (PDF via S3); PostgreSQL — system of record.

HLD — Request & Data Flow

- Booking confirmed/completed → **Accrual Worker** creates a billable item.
- Finance runs invoicing → **Invoicing Service** applies billing rules + tax → invoice (PDF via Notification).
- Payment received → **Payment Service** records it against the invoice.
- **Reconciliation Service** matches payments to invoices; flags mismatches.
- Corrections → issue a **credit note**; revenue/aging reports updated.

Scaling & performance. Scales by: Accrual is event-driven; reporting from replicas.

HLD — Technology

Layer	Technology / choice
Frontend	Next.js 14 (App Router), JavaScript
Backend	Spring Boot 3.3, Java 21 (virtual threads for I/O)
Database	PostgreSQL — Flyway migrations (ddl-auto=none)
Cache / locks	Redis (cluster) — cache, locks, pub/sub
Object storage	AWS S3 + IAM (presigned, time-limited)
Auth	JWT RS256 + dynamic MODULE:ACTION RBAC
Delivery	Docker + Swarm/K8s, GitHub Actions CI/CD

LLD — Entities & Fields

PostgreSQL types; primary keys (PK), foreign keys (FK) and indexes (IDX) annotated.

contract

A commercial contract with an organisation.

Field	Type	Notes
contract_id	uuid	PK
org_id	uuid	FK, IDX
currency	varchar(3)	ISO 4217
tax_pct	numeric(5,2)	
starts_on	date	
ends_on	date	nullable
status	varchar(16)	ACTIVE EXPIRED
created_at	timestampz	

billing_rule

A pricing rule mapped to a contract.

Field	Type	Notes
rule_id	uuid	PK
contract_id	uuid	FK, IDX
basis	varchar(16)	PER_SESSION PER_HEAD FIXED
rate	numeric(12,2)	
domain_id	uuid	FK, nullable
active	boolean	

billable_item

An accrued charge from a confirmed/completed booking.

Field	Type	Notes
item_id	uuid	PK
contract_id	uuid	FK
booking_id	uuid	FK, UNIQUE
description	varchar(200)	

quantity	numeric(10,2)	
unit_rate	numeric(12,2)	
amount	numeric(14,2)	
status	varchar(16)	ACCRUED INVOICED
accrued_at	timestamptz	

invoice

An issued invoice (immutable once issued).

Field	Type	Notes
invoice_id	uuid	PK
number	varchar(32)	UNIQUE
org_id	uuid	FK, IDX
contract_id	uuid	FK
subtotal	numeric(14,2)	
tax	numeric(14,2)	
total	numeric(14,2)	
currency	varchar(3)	
status	varchar(16)	DRAFT ISSUED PAID PARTIAL VOID
issued_at	timestamptz	nullable
due_at	timestamptz	nullable
pdf_s3_key	varchar(255)	

invoice_line

A line on an invoice (from a billable item).

Field	Type	Notes
line_id	uuid	PK
invoice_id	uuid	FK, IDX
item_id	uuid	FK → billable_item
description	varchar(200)	
quantity	numeric(10,2)	
unit_rate	numeric(12,2)	
amount	numeric(14,2)	

payment

A payment recorded against an invoice.

Field	Type	Notes
payment_id	uuid	PK
invoice_id	uuid	FK, IDX
amount	numeric(14,2)	
currency	varchar(3)	
method	varchar(16)	BANK CARD WIRE
reference	varchar(120)	
received_at	timestamptz	
recorded_by	uuid	FK

reconciliation

A reconciliation record between payments and an invoice.

Field	Type	Notes
recon_id	uuid	PK
invoice_id	uuid	FK, UNIQUE
matched_amount	numeric(14,2)	

variance	numeric(14,2)	invoice total - matched
status	varchar(16)	MATCHED MISMATCH PARTIAL
reconciled_at	timestamptz	
reconciled_by	uuid	FK

credit_note

A correction/adjustment against an invoice.

Field	Type	Notes
credit_id	uuid	PK
invoice_id	uuid	FK, IDX
amount	numeric(14,2)	
reason	varchar(255)	
issued_at	timestamptz	
issued_by	uuid	FK

LLD — Service Methods

ContractService

```
Contract createContract(ContractForm f)
```

Creates an organisation contract with currency and tax.

```
BillingRule addRule(ContractId c, RuleForm r)
```

Adds a per-session/per-head/fixed billing rule.

AccrualWorker

```
void onBookingConfirmed(BookingEvent e)
```

Creates a billable item by applying the matching billing rule to the booking.

InvoicingService

```
Invoice generate(OrgId o, Period p)
```

Aggregates ACCRUED items, applies tax, creates a DRAFT invoice with lines.

```
void issue(InvoiceId id)
```

Issues the invoice (immutable), renders a PDF and emails it via Notification.

```
CreditNote credit(InvoiceId id, BigDecimal amount, String reason)
```

Issues a credit note against an invoice.

PaymentService

```
Payment record(InvoiceId id, PaymentForm f)
```

Records a full/partial payment and updates invoice status.

ReconciliationService

```
Reconciliation reconcile(InvoiceId id)
```

Matches recorded payments to the invoice total; flags variance/mismatch.

LLD — API Endpoints

Method & path	Purpose	RBAC scope
POST /api/v1/finance/contracts	Create a contract.	FIN:CONTRACT:MANAGE

POST /api/v1/finance/contracts/{id}/rules	Add a billing rule.	FIN: CONTRACT:MANAGE
POST /api/v1/finance/invoices	Generate an invoice.	FIN: INVOICE:MANAGE
POST /api/v1/finance/invoices/{id}/issue	Issue + email an invoice.	FIN: INVOICE:MANAGE
POST /api/v1/finance/invoices/{id}/payments	Record a payment.	FIN: PAYMENT:MANAGE
POST /api/v1/finance/invoices/{id}/reconcile	Reconcile payments.	FIN: RECON:MANAGE
POST /api/v1/finance/invoices/{id}/credit-notes	Issue a credit note.	FIN: INVOICE:MANAGE
GET /api/v1/finance/revenue	Revenue & aging report.	FIN: VIEW

LLD — Key Logic & Algorithms

Accrue → invoice → reconcile

Revenue flows deterministically from confirmed delivery to reconciled payment.

- On booking-confirmed/completed, accrue a billable item using the contract's billing rule.
- At billing time, aggregate ACCRUED items for the org/period, apply tax → DRAFT invoice with lines.
- Issue the invoice (now immutable); render PDF; email via Notification.
- Record payments against the invoice; ReconciliationService matches and flags any variance.
- Corrections are credit notes, never edits to an issued invoice.

LLD — State / Lifecycle

From → To	Trigger	Effect
DRAFT → ISSUED	Finance issues	Invoice immutable; PDF emailed.
ISSUED → PARTIAL	Partial payment recorded	Outstanding balance remains.
ISSUED/PARTIAL → PAID	Payment completes total	Reconciliation MATCHED.
ISSUED → VOID	Voided (with credit note)	Adjustment recorded; not deleted.

LLD — Non-Functional Targets

Attribute	Target / approach
Integrity	Issued invoices immutable; all mutations audited.
Accuracy	Billing applies contract rules + tax deterministically.
Security	Finance scopes enforced; PII/commercial data protected.
Scalability	Accrual is event-driven; reporting from replicas.